

Dual-arm cooperative manipulation based on differential evolution

Jesús Hernández-Barragán¹, Carlos López-Franco,
Alma Y Alanis², Nancy Arana-Daniel and Michel López-Franco

Abstract

Cooperative manipulation in dual-arm robotic systems is a fundamental capability to perform many problems such as human-like tasks. In this article, we present an approach to solve dual-arm cooperative manipulation tasks using the differential evolution algorithm. In this work, manipulator kinematics are represented using the Denavit–Hartenberg model. The proposed method is able to avoid singularities because it does not require the inversion of any Jacobian matrix. In addition, the proposed approach handles joint limit constraints based on penalty functions. As a final remark, this approach is suitable for robotic systems of redundant or non-redundant serial manipulators composed of revolute or prismatic joints. Simulation experiments illustrate the effectiveness of the proposed approach under different dual-arm configurations. Furthermore, real experiments were performed using a dual-arm KUKA Youbot system to show the applicability of the proposed approach.

Keywords

Meta-heuristics algorithms, differential evolution, path tracking, dual-arm manipulation, cooperative manipulators

Date received: 10 September 2018; accepted: 14 December 2018

Topic: Field Robotics

Topic Editor: Andrey V Savkin

Associate Editor: Istvan Harmati

Introduction

Dual-arm systems can perform different tasks such as handling large and heavy objects and perform complex cooperative tasks.¹ These manipulators are also used to solve human-like tasks in both domestic and industrial environments.² The term dual-arm manipulation can be defined as a physical interaction with an object, exerting forces on it to move or reshape it using two robot manipulators.² However, there is no specific agreed definition for this term. In general, dual-arm systems can be classified into non-coordinated manipulation and coordinated manipulation. In non-coordinated manipulation, arms performed different tasks independently, therefore their analysis does not need to differ from single-arm systems. In contrast, in coordinated manipulation, arms are physically interacting with each other, for this reason, their relative motions have to be adequately controlled to perform a task. In this work, we

proposed to solve the dual-arm cooperative manipulation for coordinate manipulation tasks.

An accepted method to maintain coordinate manipulation of dual-arm systems is the relative Jacobian matrix.^{3,4} This method allows to considerate dual-arm systems as an unique redundant manipulator, where its end-effector motion variables correspond to the relative motion between the two grippers, and its number of joints is equal to the sum of the joints relative to each manipulator. Moreover, a

Department of Computer Science, University of Guadalajara, Guadalajara, Jalisco, México

Corresponding author:

Jesús Hernández-Barragán, Department of Computer Science, University of Guadalajara, CUCEI, Marcelino García Barragán 1421, Guadalajara cp 44100, México.

Email: jesus.hdez.barragan@gmail.com



Creative Commons CC BY: This article is distributed under the terms of the Creative Commons Attribution 4.0 License (<http://www.creativecommons.org/licenses/by/4.0/>) which permits any use, reproduction and distribution of the work without further permission provided the original work is attributed as specified on the SAGE and Open Access pages (<https://us.sagepub.com/en-us/nam/open-access-at-sage>).

dual-arm system modeled by the relative Jacobian matrix can be controlled by the same algorithm used for single-arm systems. Then, to perform a cooperative task, the relative Jacobian matrix method is extended with the Jacobian null-space protection.^{4,5} This approach allows to perform multiple tasks. Commonly, the highest priority task is set to the relative motion between the two grippers. Then, a secondary task is given by a desired end-effector pose with respect to the main manipulator. Other tasks can be obstacle avoidance or handle joint limits constraints. We called this approach the conventional method. However, an inconvenience of this approach is the task conflicts given by singularities. Singularities occur when the Jacobians for different tasks become rank deficient.⁶ That is because they require the inversion of the Jacobian matrix, then the inverse kinematics task fails. To overcome this inconvenience, we propose to represent manipulators kinematics using the Denavit–Hartenberg (DH) model.⁷ Then, an objective function is formulated based on the forward kinematics equations and it does not need the inclusion of any Jacobian matrix. Some authors have proposed different method to handle properly the conflicts with tasks priorities.^{1,6} However, those methods still require the inversion of a Jacobian matrix. Additionally, the selection of an initial joint configuration is a key problem. At some particular initial joint configuration, the inverse kinematics task may fail. In consequence, abrupt changes in joint values can be presented because of singularities.

Redundancy in robotic manipulators is a very important problem to solve. Generally, a single-arm system is defined as kinematically redundant when its degree of motion n (number of joints) is higher than the number of variables m that are necessary to perform a given task (dimension of the task), this is $n > m$.⁵ Since a dual-arm manipulation of a rigid object form a closed kinematic chain, then dual-arm systems are considered kinematically redundant.^{1,8} Redundancy solutions admit several joints configuration to reach the same desired end-effector pose, in consequence, the inverse kinematics becomes difficult to solve. A convenient approach to solve this problem is global optimization of a specific objective function. However, many local minimum may be presented and the optimization problem becomes challenging to solve. For this reason, it is necessary to search for no conventional methods to solve this problem. Meta-heuristic algorithms have been applied successfully to solve many optimization problems in different research areas.⁹ For single-arm manipulator, several meta-heuristic algorithms have solved the inverse kinematics and path tracking problem for redundant and non-redundant manipulators.^{10–14} A hybrid algorithm between differential evolution (DE) and modified particle swarm optimization (MPSO) has been proposed to solve the inverse kinematics of serial manipulators¹⁵ and parallel manipulators.¹⁶ This method is called DEMPSO and it performed better than DE and MPSO versions. Ren et al.¹⁷ present a hybrid approach for solving the inverse kinematics of a serial manipulator

based on DE and biogeography-based optimization (BBO). The approach is called hybrid biogeography-based optimization (HBBO). The authors included a comparative study among DE, BBO, genetic algorithms (GA), and HBBO, where HBBO reported better results than the other algorithms. Then, the imperialist competitive algorithm (ICA) was also applied to solve the inverse kinematics of serial manipulators.¹⁸ In this case, it is shown a comparison among DE, PSO, ICA, harmony search (HS), and cuckoo optimization algorithm (COA). The authors concluded that ICA solved the given problems accurately with fast convergence rate than the other algorithms. Finally, an improved version of teaching-learning based optimization (TLBO) has been proposed to solve the optimal trajectory planning of robot manipulators.¹⁹ The method is called ITLBO and it proves to be superior than several version of TLBO for solving the given tasks. In this work, we propose to use DE to solve the dual-arm cooperative manipulation as a global constrained optimization problem. DE is a simple but powerful optimization algorithm. We consider that a standard version of DE is enough to solve dual-arm cooperative task better or similar than those meta-heuristics algorithms mentioned before. Constraints in dual-arm manipulation are given by joint limits. There are many techniques to solve constraints optimization problems.²⁰ However, in this approach, we propose to use penalty functions to solve this problem.

In this article, we proposed to solve the dual-arm cooperative manipulation tasks as a global constrained optimization problem. The contributions of the article are given below: the proposed approach is able to avoid singularities. Since the dual-arm system is modeled using the DH convention, we use the forward kinematics equations to formulate an objective function. Because forward kinematics always have a solution, it is not required the inversion of any Jacobian matrix. Moreover, the proposed approach handles joint limit constraints. To handle constraints, we propose to use penalty functions by adding penalties to the proposed objective function. Additionally, the proposed approach can be used for dual-arm systems of redundant or non-redundant serial manipulators composed of revolute or prismatic joints.

This work is organized as follows: the next section gives a comprehensive description about the proposed approach where we include a description about dual-arm system model, DE algorithm, and the objective function is formulated in detail. Then, simulations results are given where we present inverse kinematics results under different dual-arm systems. Comparative studies are also included here. Next, the experiment results are reported where a dual-arm KUKA Youbot system was considered for real experiments. Finally, we give the conclusions.

Methodology

In this work, we propose to solve the dual-arm cooperative manipulation as a global constrained optimization problem

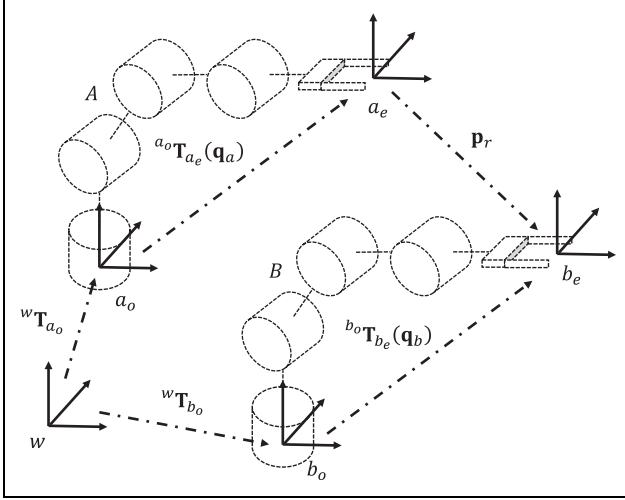


Figure 1. Kinematic model of a dual-arm system composed by two manipulators A and B.

of a specific objective function. Here, we describe a kinematic model for dual-arm systems based on the DH model. Then, an objective function is proposed using its forward kinematics equations. To handle joint limits constraints, we use penalty functions to penalize those joint values that violated the allowed joint restrictions. Finally, to solve the constrained optimization problem, we proposed to use the DE algorithm. A detailed description of the proposed approach is given in the next subsections.

Dual-arm system model

Figure 1 shows a dual-arm system composed by two manipulators A and B with a total number of joints n_a and n_b , respectively. Then, we define a joint variable $\mathbf{q}_a \in \mathbb{R}^{n_a}$ and $\mathbf{q}_b \in \mathbb{R}^{n_b}$ for manipulators A and B, respectively. For manipulator A, the transformation of the end-effector frame a_e with respect to its base frame a_o is given by the homogeneous matrix ${}^{a_o}\mathbf{T}_{a_e}(\mathbf{q}_a)$. Similarly, the transformation of the end-effector frame b_e with respect to the base frame b_o is given by the homogeneous matrix ${}^{b_o}\mathbf{T}_{b_e}(\mathbf{q}_b)$, for manipulator B.

For a cooperative manipulation task, manipulator A is considered to have the role master. Then, the end-effector position of frame b_e is expressed with respect to the end-effector position of frame a_e using the vector $\mathbf{p}_r$. Indeed, $\mathbf{p}_r$ defines the relative position between the end-effector position of manipulators A and B.

The end-effector pose ${}^w\mathbf{T}_{a_e}$ of manipulator A and ${}^w\mathbf{T}_{b_e}$ of manipulator B, with respect to a world frame w , can be computed as follows

$$\begin{aligned} {}^w\mathbf{T}_{a_e} &= {}^w\mathbf{T}_{a_o} {}^{a_o}\mathbf{T}_{a_e}(\mathbf{q}_a) = \begin{bmatrix} \mathbf{R}_{a_e} & \mathbf{p}_{a_e} \\ \mathbf{0} & 1 \end{bmatrix} \\ {}^w\mathbf{T}_{b_e} &= {}^w\mathbf{T}_{b_o} {}^{b_o}\mathbf{T}_{b_e}(\mathbf{q}_b) = \begin{bmatrix} \mathbf{R}_{b_e} & \mathbf{p}_{b_e} \\ \mathbf{0} & 1 \end{bmatrix} \end{aligned} \quad (1)$$

where $\mathbf{R}_{a_e}$ and $\mathbf{p}_{a_e}$ define the orientation and position of the end-effector frame a_e , respectively. Finally, $\mathbf{R}_{b_e}$ and $\mathbf{p}_{b_e}$ define the orientation and position of the end-effector frame b_e , respectively.

Objective function formulation

For convenience, we define the actual joints configuration of a dual-arm system as $\mathbf{q} = [\mathbf{q}_a \quad \mathbf{q}_b]^T$ where $\mathbf{q} \in \mathbb{R}^{n_a+n_b}$. Then, from $\mathbf{q}$, we can compute the actual end-effector position $\mathbf{p}_{a_e}$ and $\mathbf{p}_{b_e}$ with respect to the world frame w using equation (1). We perform this operation based on the kinematic model of each manipulator obtained from DH model.⁷

Previously, we define a position error between a desired end-effector position $\mathbf{p}_{a_e}^*$ and the actual end-effector position $\mathbf{p}_{a_e}$, for manipulator A as follows

$$p_{a_{\text{error}}} = \|\mathbf{p}_{a_e}^* - \mathbf{p}_{a_e}\| \quad (2)$$

with respect to manipulator B, the position error between the desired end-effector position $\mathbf{p}_{b_e}^*$ and the actual end-effector position $\mathbf{p}_{b_e}$ is given by

$$p_{b_{\text{error}}} = \|\mathbf{p}_{b_e}^* - \mathbf{p}_{b_e}\| \quad (3)$$

where $\mathbf{p}_{b_e}^* = \mathbf{p}_{a_e}^* + \mathbf{p}_r$, and $\mathbf{p}_r$ is the relative position between the end-effector position of manipulators A and B.

To minimize the cooperative manipulation motion, we define an error between the actual joints configuration $\mathbf{q}$ and the previous joints configuration $\mathbf{q}_{\text{old}}$. We define this error as

$$q_{\text{error}} = \|\mathbf{q}_{\text{old}} - \mathbf{q}\| \quad (4)$$

Then, we formulate an objective function with the weighted sum of errors given in equations (2) to (4) as follows

$$f = \alpha p_{a_{\text{error}}} + \beta p_{b_{\text{error}}} + \gamma q_{\text{error}} \quad (5)$$

where α , β , and γ are scalar factors that scale the contribution of each error. Since the proposed approach solves the dual-arm cooperative manipulation as a constrained optimization problem, it is needed to include constraints in equation (5).

The lower and upper joint limits for manipulator A are defined as $\mathbf{q}_{a_l}$ and $\mathbf{q}_{a_u}$, respectively. Similarly, $\mathbf{q}_{b_l}$ and $\mathbf{q}_{b_u}$ define the lower and upper joint limits for manipulator B. Then, vectors $\mathbf{q}_l = [\mathbf{q}_{a_l} \quad \mathbf{q}_{b_l}]^T$ and $\mathbf{q}_u = [\mathbf{q}_{a_u} \quad \mathbf{q}_{b_u}]^T$ with $\mathbf{q}_l, \mathbf{q}_u \in \mathbb{R}^{n_a+n_b}$ are defined to represent the lower and upper joint limits for the dual-arm system, respectively.

Particularly, it is necessary to penalize a joint configuration q_i when its value is smaller than the allowed lower joint limit q_{l_i} . Moreover, the value of joint q_i can also be larger than the allowed upper joint limit q_{u_i} . To solve these two inconveniences, we propose to define two penalty functions. The penalty function defined in equation (6)

handles constraints when $q_{li} < q_i$. On the other hand, the penalty function given in equation (5) handles constraints when $q_i < q_{ui}$.

$$g(q_i) = \begin{cases} 0 & \text{if } q_{li} < q_i \\ (q_i - q_{li})^2 & \text{otherwise} \end{cases} \quad (6)$$

$$h(q_i) = \begin{cases} 0 & \text{if } q_i < q_{ui} \\ (q_i - q_{ui})^2 & \text{otherwise} \end{cases} \quad (7)$$

Finally, we include the two defined penalty functions given in equations (6) and (7), into equation (5) to obtain the proposed objective function f' that handles joint limits constraints as

$$f' = f + \sum_{i=1}^{n_a+n_b} \delta[g(q_i) + h(q_i)] \quad (8)$$

where δ scales the penalization term that is usually selected as a large constant. We recommend to use $\delta = 1000$.

To conclude, the optimal joint configuration $\mathbf{q}$ is found by solving the constrained optimization problem defined as

$$\begin{aligned} & \min_{\mathbf{q}} f'(\mathbf{q}), \\ & \text{subject to } \mathbf{q}_l < \mathbf{q} < \mathbf{q}_u \end{aligned}$$

Differential evolution

DE is a population-based optimization algorithm, where every individual in the population represent a candidate solution. Individuals improve during an iterative process by performing three principal operations: mutation, crossover, and selection.

The algorithm begins with the initialization of individuals $\mathbf{x}_i$ with $i = 1, 2, 3, \dots, N$, where N is the total number of population members. In the mutation operation, a mutant vector $\mathbf{v}_i$ is generated for each individual i as follows

$$\mathbf{v}_i = \mathbf{x}_{r_1} + F(\mathbf{x}_{r_2} - \mathbf{x}_{r_3}) \quad (9)$$

where $\mathbf{x}_{r_1}$, $\mathbf{x}_{r_2}$, and $\mathbf{x}_{r_3}$ are individuals chosen randomly such as $r_1, r_2, r_3 \in [1, N]$ and $i \neq r_1 \neq r_2 \neq r_3$. The amplification factor $F \in [0, 2]$ controls the contribution of the difference $(\mathbf{x}_{r_2} - \mathbf{x}_{r_3})$. Then, a trial vector $\mathbf{u}_i$ is created by the following crossover operation

$$u_{ij} = \begin{cases} v_{ij} & \text{if } r_j \leq C_R \text{ or } j = r_i \\ x_{ij} & \text{if } r_j > C_R \text{ and } j \neq r_i \end{cases} \quad (10)$$

where $C_R \in [0, 1]$ is the crossover constant and $j = 1, 2, 3, \dots, N$ with D as the problem dimension. Vales r_j and r_i are random numbers defined as $r_j \in [0, 1]$ and $r_i \in [1, N]$. Finally, in the selection operation, the trial vector $\mathbf{u}_i$ is compared to the actual vector $\mathbf{x}_i$ in the evaluation of an objective function f . If $f(\mathbf{u}_i)$ yields a better solution than $f(\mathbf{x}_i)$, then $\mathbf{u}_i$ replaces $\mathbf{x}_i$, otherwise, $\mathbf{x}_i$ is retained to the

Algorithm 1. Cooperative inverse kinematics algorithm.

```

1:  $f' \leftarrow$  objective function defined in equation (8)
2: set  $N, F \in [0, 2]$  and  $C_R \in [0, 1]$  values
3: for each individual  $j$  do
4:   initialize individual position  $\mathbf{q}_j$  using equation (11)
5: end
6: repeat
7:   for each individual  $j$  do
8:     randomly choose  $\mathbf{q}_{r_1}$ ,  $\mathbf{q}_{r_2}$ , and  $\mathbf{q}_{r_3}$ 
9:     compute a mutant vector  $\mathbf{v}_j$  using equation (9)
10:    calculate a trial vector  $\mathbf{u}_j$  using equation (10)
11:    if  $f'(\mathbf{u}_j) < f'(\mathbf{q}_j)$  then
12:       $\mathbf{q}_j \leftarrow \mathbf{u}_j$ 
13:    end
14:  end
15: until stop criteria met
16: report results

```

next iteration. A detailed description about DE and its modification can be found in the study by Storn and Price.²¹

Proposed approach description

In this work, we propose to solve the dual-arm manipulation problem using DE algorithm. We consider to solve cooperative inverse kinematics tasks and cooperative path tracking tasks. To solve the cooperative tasks, we have to define the transformations ${}^w\mathbf{T}_{a_0}$ and ${}^w\mathbf{T}_{b_0}$ for manipulators A and B , respectively. The relative position $\mathbf{p}_r$ between A and B also have to be defined. Joints limits $\mathbf{q}_l$ and $\mathbf{q}_u$ must be provided. The vector $\mathbf{q}_{old}$ is chosen as the initial joint configuration. Finally, the parameter setting for the objective function α , β , and γ needs to be provided. We recommend to use $\alpha = \beta = 1.0$ and $\gamma = 0.02$.

The algorithm of the proposed approach for solving the inverse kinematics of a dual-arm system is given in Algorithm 1. In this algorithm, every individual j contains a set of candidate joint solutions $\mathbf{q}_j$. Then, the initialization of each individual j is given by

$$\mathbf{q}_j = \mathbf{q}_l + (\mathbf{q}_u - \mathbf{q}_l) r \quad (11)$$

where $r \in [0, 1]$ is a random number, $\mathbf{q}_l$ and $\mathbf{q}_u$ are the lower and upper joint limits.

The stop criteria is met when DE algorithm achieves the total number of iterations or when the evaluation of the fitness function reaches the allowed tolerance s_{tol} . In this case, the allowed tolerance occurs when $s_{tol} < s_{stop}$, where $s_{stop} = p_{a_{error}} + p_{b_{error}}$ which is the error of the end-effector position of manipulators A and B .

For a cooperative path tracking task, the path is divided into $k = 1, 2, 3, \dots, M$ points, where M is the total number of points in the path. Then, for each point $\mathbf{p}_k^*$, the inverse kinematics problem is solved. The algorithm of the proposed approach for solving a cooperative path tracking problem of a dual-arm system is given in Figure 2.

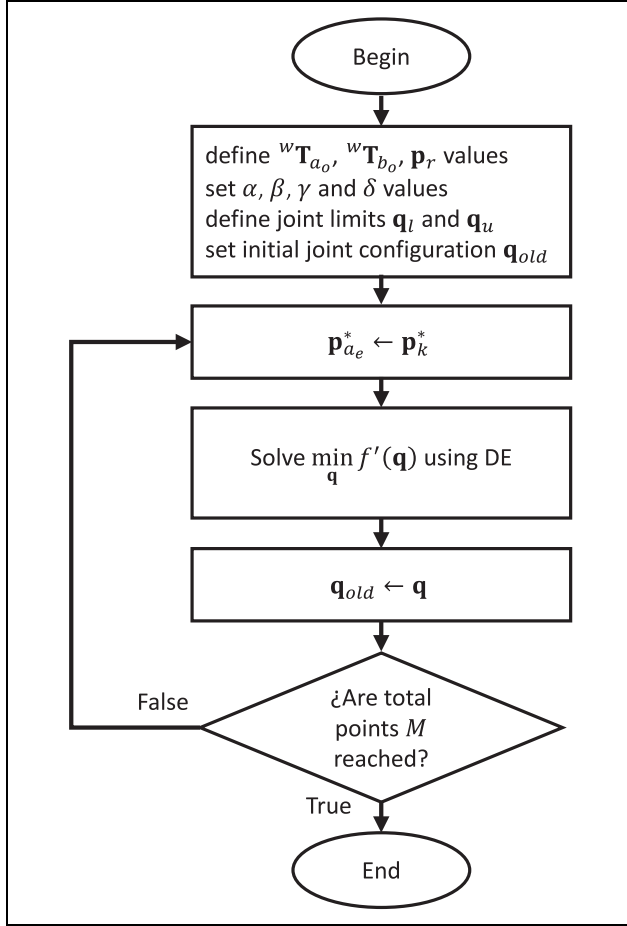


Figure 2. Cooperative path tracking algorithm. DE: differential evolution.

Table 1. DH table for SCARA manipulator.

Joint	a (mm)	α (rad)	d (mm)	θ (rad)
1	250	0	400	q_1
2	150	π	0	q_2
3	0	0	q_3	0
4	0	0	75	q_4

DH: Denavit–Hartenberg.

Simulation results

The aim of simulations is to test the behavior of the proposed approach for solving the dual-arm cooperative manipulation tasks of different dual-arm systems. The simulations were conducted as follows: First, we performed several cooperative inverse kinematics tasks using different dual-arm systems to test the proposed approach. In this test, we presented a comparative study to compare the performance of DE, DEMPSO, HBBO, ICA, and ITLBO. Then, we included non-parametric statistical test to detect a significant difference among the considered meta-heuristic algorithms under the inverse kinematics

Table 2. DH table for KUKA Youbot manipulator.

Joint	a (mm)	α (rad)	d (mm)	θ (rad)
1	33	$\pi/2$	147	q_1
2	155	0	0	q_2
3	135	0	0	q_3
4	0	$\pi/2$	0	q_4
5	0	0	217.5	q_5

DH: Denavit–Hartenberg.

Table 3. DH table for Puma 560 manipulator.

Joint	a (mm)	α (rad)	d (mm)	θ (rad)
1	0	$\pi/2$	0	q_1
2	431.8	0	0	q_2
3	20.3	$-\pi/2$	150	q_3
4	0	$\pi/2$	431.8	q_4
5	0	$-\pi/2$	0	q_5
6	0	0	0	q_6

DH: Denavit–Hartenberg.

Table 4. DH table for Stanford manipulator.

Joint	a (mm)	α (rad)	d (mm)	θ (rad)
1	0	$-\pi/2$	412	q_1
2	0	$\pi/2$	154	q_2
3	0	0	q_3	0
4	0	$-\pi/2$	0	q_4
5	0	$\pi/2$	0	q_5
6	0	0	263	q_6

DH: Denavit–Hartenberg.

Table 5. DH table for Baxter manipulators.

Joint	a (mm)	α (rad)	d (mm)	θ (rad)
1	69	$-\pi/2$	270.35	q_1
2	0	$\pi/2$	0	q_2
3	69	$-\pi/2$	364.35	q_3
4	0	$\pi/2$	0	q_4
5	10	$-\pi/2$	374.29	q_5
6	10	$\pi/2$	0	q_6
7	10	0	229.525	q_7

DH: Denavit–Hartenberg.

test. Finally, we performed cooperative path tracking tasks to compare the performance of the proposed approach against the conventional method. In the simulations, we test the proposed approach under dual-arm systems composed of different manipulator structures. The robots considered in this work are the SCARA, KUKA Youbot, Puma 560, Stanford, and Baxter, and their DH tables are given in Tables 1 to 5, respectively (The KUKA Youbot and Baxter robots are located in the Intelligent Systems Laboratory at the University of Guadalajara. The

Puma 560, Stanford and SCARA robots are often used as an educational example in robot kinematics, they were only implemented for simulations.).

The joint limits constraints for a manipulator structure are commonly defined by the joint motors specifications. However, for simulations, the users can provide specific joints limits. The proposed joint limits are the following: The lower $\mathbf{q}_l$ and upper $\mathbf{q}_u$ joints limits of SCARA manipulator are set to

$$\mathbf{q}_l = [-2.1817 \quad -2.5307 \quad 0 \quad -3.1241]^T$$

$$\mathbf{q}_u = [2.1817 \quad 2.5307 \quad 0.3000 \quad 3.1241]^T$$

The lower and upper joints limits of KUKA Youbot manipulator are fixed as follows

$$\mathbf{q}_l = [-2.9496 \quad -1.1345 \quad -2.6180 \quad -1.7837 \quad -2.9234]^T$$

$$\mathbf{q}_u = [2.9496 \quad 1.5708 \quad 2.5482 \quad 1.7890 \quad 2.9234]^T$$

The joints limits of Puma 560 manipulator are selected as

$$\mathbf{q}_l = [-2.7925 \quad -2.1817 \quad -2.3562 \quad -2.6180 \quad -1.7453 \quad -4.6426]^T$$

$$\mathbf{q}_u = [2.7925 \quad 2.1817 \quad 2.3562 \quad 2.6180 \quad 1.7453 \quad 4.6426]^T$$

The lower and upper joints limits of Stanford manipulator are given as

$$\mathbf{q}_l = [-2.6180 \quad -1.5708 \quad 0.1000 \quad -2.7925 \quad -0.4363 \quad -2.7925]^T$$

$$\mathbf{q}_u = [2.6180 \quad 1.5708 \quad 0.6000 \quad 2.7925 \quad 0.4363 \quad 2.7925]^T$$

Finally, joints limits of Baxter manipulator are set to

$$\mathbf{q}_l = [-1.7017 \quad -2.1468 \quad -3.0543 \quad -0.0524 \quad -3.0543 \quad -1.5708 \quad -3.0543]^T$$

$$\mathbf{q}_u = [1.7017 \quad 1.0472 \quad 3.0543 \quad 2.6180 \quad -3.0543 \quad 2.0944 \quad 3.0543]^T$$

Cooperative inverse kinematics test

In this test, we performed several cooperative inverse kinematics tasks to test the performance of the proposed approach. We proposed to use four dual-arm systems with different kinematic structures. The relative position $\mathbf{p}_r$ of the end-effector of manipulator B with respect to the end-effector of manipulator A has to be defined by the user to perform a cooperative task. The transformations ${}^w\mathbf{T}_{a_o}$ and ${}^w\mathbf{T}_{b_o}$ define the base frame of manipulators A and B with respect to the world frame, respectively. These transformations also have to be defined by the user. Finally, it is necessary to provide an initial joint configuration $\mathbf{q}_{old}$ for

the dual-arm system. The considered dual-arm systems are given below.

The first dual-arm system is composed of two KUKA Youbot manipulators based on DH parameters from Table 2. Their transformations ${}^w\mathbf{T}_{a_o}$ and ${}^w\mathbf{T}_{b_o}$ were fixed as

$${}^w\mathbf{T}_{a_o} = \begin{bmatrix} 1 & 0 & 0 & 0.0 \\ 0 & 1 & 0 & 0.151 \\ 0 & 0 & 1 & 0.140 \\ 0 & 0 & 0 & 1 \end{bmatrix}, {}^w\mathbf{T}_{b_o} = \begin{bmatrix} 1 & 0 & 0 & 0.0 \\ 0 & 1 & 0 & -0.151 \\ 0 & 0 & 1 & 0.140 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The relative position $\mathbf{p}_r$ between the end-effector of manipulators A and B was selected as $\mathbf{p}_r = [0.0 \quad -0.2 \quad 0.0]^T$. The initial joint configuration was set to

$$\mathbf{q}_{old} = [0 \quad 1.5708 \quad 0 \quad 1.5708 \quad 0 \quad 0 \quad 1.5708 \quad 0 \quad 1.5708 \quad 0]^T$$

The second dual-arm system is composed of two Stanford manipulators, see Table 4 for DH Table. Their transformations ${}^w\mathbf{T}_{a_o}$ and ${}^w\mathbf{T}_{b_o}$ were selected as

$${}^w\mathbf{T}_{a_o} = \begin{bmatrix} 1 & 0 & 0 & 0.5 \\ 0 & 1 & 0 & 0.6 \\ 0 & 0 & 1 & 0.0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, {}^w\mathbf{T}_{b_o} = \begin{bmatrix} 1 & 0 & 0 & -0.5 \\ 0 & 1 & 0 & -0.6 \\ 0 & 0 & 1 & 0.0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The relative position $\mathbf{p}_r$ was set to $\mathbf{p}_r = [0.3 \quad -1.0 \quad 0.0]^T$ and the initial joint configuration was given by

$$\mathbf{q}_{old} = [0 \quad 0 \quad 0.1000 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0.1000 \quad 0 \quad 0 \quad 0]^T$$

The third dual-arm system is composed of two Baxter manipulators based on Table 5. Transformations ${}^w\mathbf{T}_{a_o}$ and ${}^w\mathbf{T}_{b_o}$ were defined as

$${}^w\mathbf{T}_{a_o} = \begin{bmatrix} 0.70711 & -0.70711 & 0 & 0.064614 \\ 0.70711 & 0.70711 & 0 & 0.25858 \\ 0 & 0 & 1 & 0.119 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^w\mathbf{T}_{b_o} = \begin{bmatrix} 0.70711 & 0.70711 & 0 & 0.063534 \\ -0.70711 & 0.70711 & 0 & -0.25966 \\ 0 & 0 & 1 & 0.119 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The relative position $\mathbf{p}_r$ was selected as $\mathbf{p}_r = [0.0 \quad -0.40 \quad 0]^T$. Then, initial joints configuration was set to

$$\mathbf{q}_{old} = [0 \quad -1.0472 \quad 0 \quad 1.0472 \quad 0 \quad 1.5708 \quad 0 \quad 0 \quad -1.0472 \quad 0 \quad 1.0472 \quad 0 \quad 1.5708 \quad 0]^T$$

Finally, the fourth dual-arm system is a hybrid combination of a Puma 560 manipulator based on DH parameters

given in Table 3 and a SCARA manipulator based on DH Table 1. Transformations ${}^w\mathbf{T}_{a_o}$ and ${}^w\mathbf{T}_{b_o}$ were defined as

$${}^w\mathbf{T}_{a_o} = \begin{bmatrix} 1 & 0 & 0 & 0.0 \\ 0 & 1 & 0 & 0.4 \\ 0 & 0 & 1 & 0.0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, {}^w\mathbf{T}_{b_o} = \begin{bmatrix} 1 & 0 & 0 & 0.0 \\ 0 & 1 & 0 & -0.4 \\ 0 & 0 & 1 & 0.0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The relative position $\mathbf{p}_r$ was given by $\mathbf{p}_r = [-0.3 \ -0.8 \ -0.2]^T$. The initial joint configuration was set to

$$\mathbf{q}_{old} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ -1.0472 & 1.7453 & 0 & 0 \end{bmatrix}^T$$

In this test, we also presented a comparative study to compare the performance of the DE, DEMPSO, HBBO, ICA, and ITLBO. The parameter setting for these algorithms is conducted as follows: for the common parameter setting, we decided to use a population size of 50 individuals, a total number of 500 iterations, and a tolerance of 0.0001. For DE, we used the standard DE with the mutation strategy DE/rand/1/bin. We also fixed the amplification factor F and the crossover constant C_R to 0.6 and 0.9, respectively. In DEMPSO, with respect to PSO settings, the cognitive factor c_1 is gradually decreased from 2.5 to 0.5, the social factor c_2 is gradually increased from 0.5 to 2.5, the weighted factor w is gradually decreased from 0.9 to 0.6, and the same DE setting are used. For HBBO, we fixed the maximum immigration rate I and the maximum emigration rate E to 1, the predetermined maximum mutation probability is $\phi_{max} = 0.05$, and the scaling factor and crossover probability of the DE strategy are $F = 0.6$ and $C_R = 1.0$. In ICA, the number of imperialist is set to 5, and the factors $\beta = 2$, $\gamma = \pi/4$, and $\xi = 0.1$. Finally for ITLBO, the threshold of grade difference ts_f is set to 0.1, the threshold of failure number ts_n is set to 2, the number of new learners in neighborhood N_{nl} is set to 10, the initial radius of the neighborhood $\delta = 0.5$, the step size to extend the neighborhood Δ is set to 0.2, and the selection times of neighborhood $N_{st} = 5$. These parameters setting are based on previous studies.^{15,17–19,22}

Simulations experiments were performed using Matlab [version 2014a 64-bit]. Each algorithm computed the cooperative inverse kinematics of 50 points calculated randomly such as $\mathbf{p}_{a_e}^* \in [\mathbf{p}_l, \mathbf{p}_u]$, where $\mathbf{p}_l$ and $\mathbf{p}_u$ define the inverse kinematics boundary. For Youbot system, boundaries were set to $\mathbf{p}_l = [0.3 \ 0.1 \ 0.3]^T$ and $\mathbf{p}_u = [0.4 \ 0.3 \ 0.5]^T$. Then, for Stanford system, boundaries were given as $\mathbf{p}_l = [0.6 \ 0.1 \ 0.3]^T$ and $\mathbf{p}_u = [0.9 \ 0.5 \ 0.6]^T$. For Baxter system, $\mathbf{p}_l = [-0.3 \ 0.3 \ 0.4]^T$ and $\mathbf{p}_u = [0.0 \ 0.706]^T$. Finally, $\mathbf{p}_l = [0.50 \ 0.40 \ 0.40]^T$ and $\mathbf{p}_u = [0.55 \ 0.50 \ 0.50]^T$ are defined for the hybrid system. We used box plots to display graphically statistical variation for the results of the proposed meta-heuristics

algorithms. In these simulations, the execution time, the position error $p_{a_{error}} + p_{b_{error}}$, and the displacement error q_{error} results are presented into box plots. We are interested in finding the algorithm with the smaller data distribution, the lower values results, and the less quantities of outliers.

Figure 3 shows the position error results for the proposed dual-arm systems. The results in Figure 3(a) correspond to the Youbot arm system. Here, every meta-heuristic algorithm presented similar results below to 0.01 mm, but DE did not present outliers results. Similarly, the results for the Stanford arms in Figure 3(b) indicate that every meta-heuristic algorithm had similar results with values under 0.01 mm of error. In the results for Baxter arms in Figure 3(c), the ITLBO algorithm presented the larger data distribution results, in contrast, DE, HBBO, and ICA show the best results that is below to 0.001 of error. The DEMPSO also shows the biggest number of outliers. Finally, Figure 3(d) shows the results for the dual-arm hybrid system. In this case, the results for every algorithm are below to 0.001 mm and they also have outliers.

Then, in Figure 4, the displacement error results for the considered dual-arm systems are reported. In the case of the Youbot arms and the hybrid arm systems presented in Figure 4(a) and Figure 4(d), respectively, there is no noticeable difference in the displacement results obtained by the considered algorithms. The algorithms performed below to an error of 2.6 in the Youbot arms and 1.1 in the hybrid arms. On the other hand, DE and ITLBO reported the smaller data distribution among the other respect to the Baxter arms in Figure 4(c). They performed below an error of 1.5. In contrast, ICA and HBBO show the larger data distribution. Finally, in Figure 4(b), the results for the Stanford arm are presented. Here, the compared algorithms performed similar with values under an error of 2.3; however, DEMPSO, HBBO, and ITLBO present outlier results.

The execution time results for the considered dual-arm systems are given in Figure 5. In this case, DE and HBBO show the smaller execution time and the smaller data distribution results against the other algorithms for the dual-arm systems. Moreover, ITLBO illustrates the bigger execution time and the larger data distribution results among the other algorithms for the dual-arm systems.

Based on the results obtained in the cooperative inverse kinematics test, DE and HBBO performed better than the other algorithms with respect to time execution. Additionally, DE and ITLBO performed better than the others in the displacement results of the Baxter arms. In contrast, there is no visual difference among the compared algorithms in the case of position error results. For this reason, we included a non-parametric statistical test to report a significant difference among the considered meta-heuristic algorithms.

Moreover, execution time results indicate that the proposed method requires so much execution time to solve the cooperative inverse kinematics tasks. It is recommended to use the proposed approach to solve the cooperative

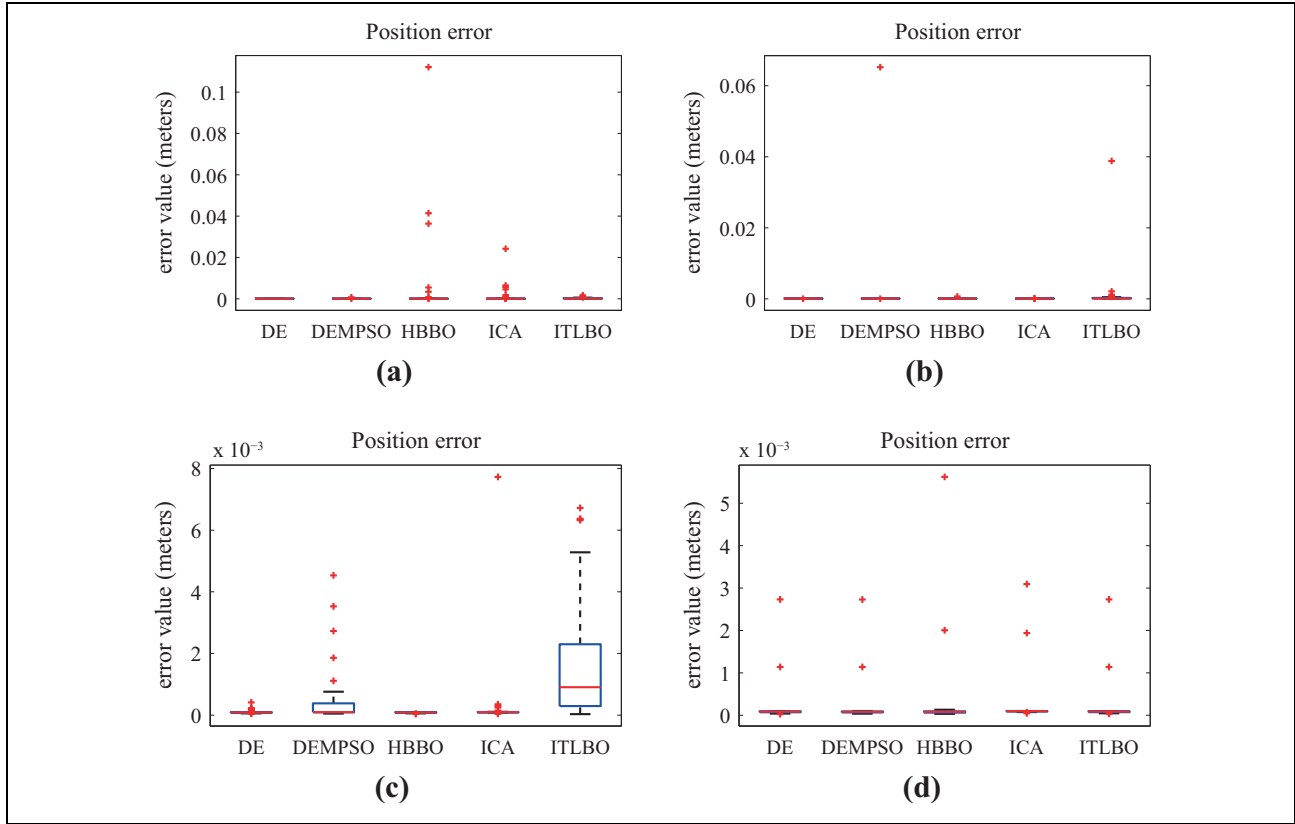


Figure 3. Position error $p_{d_{error}} + p_{b_{error}}$ results for the considered meta-heuristic algorithms. (a) Youbot arms, (b) Stanford arms, (c) Baxter arms, and (d) hybrid arms.

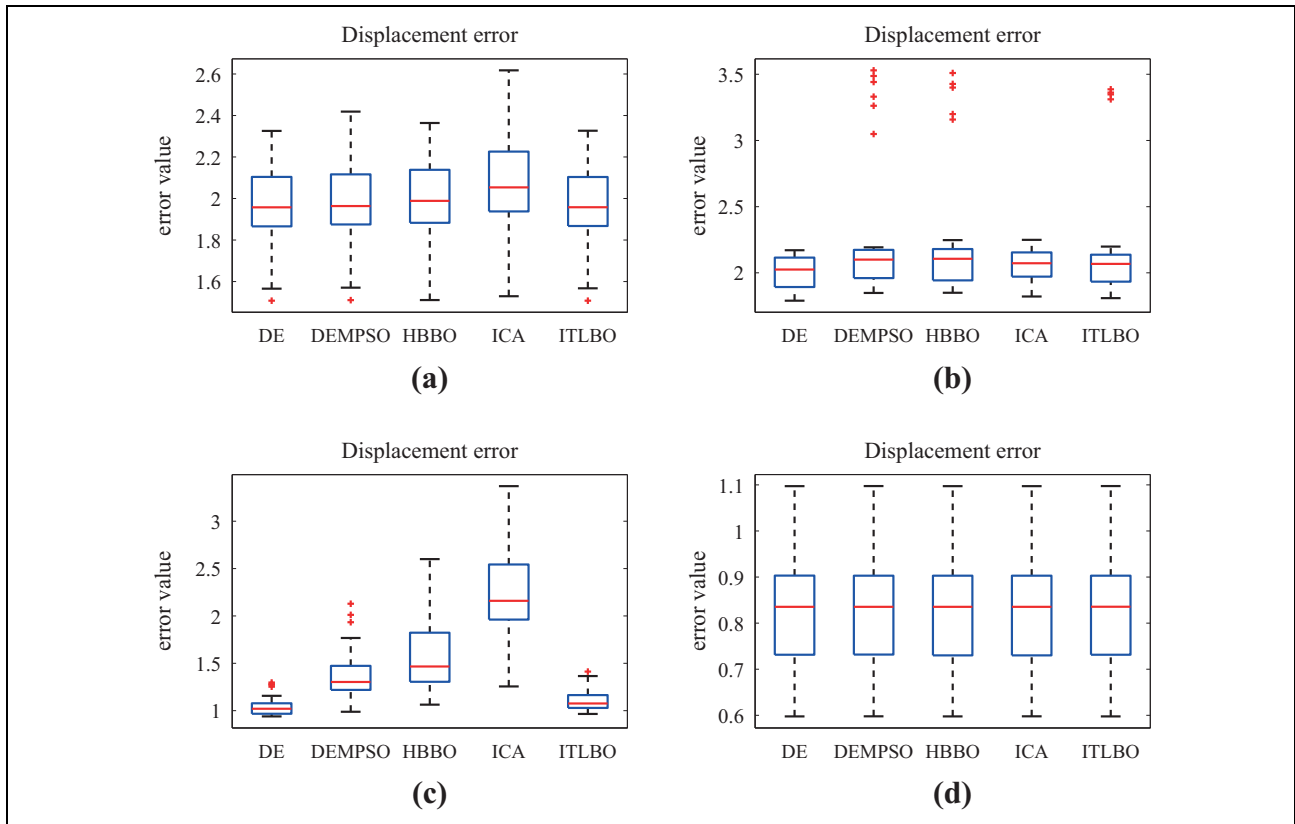


Figure 4. Displacement error q_{error} results for the considered meta-heuristic algorithms. (a) Youbot arms, (b) Stanford arms, (c) Baxter arms, and (d) hybrid arms.

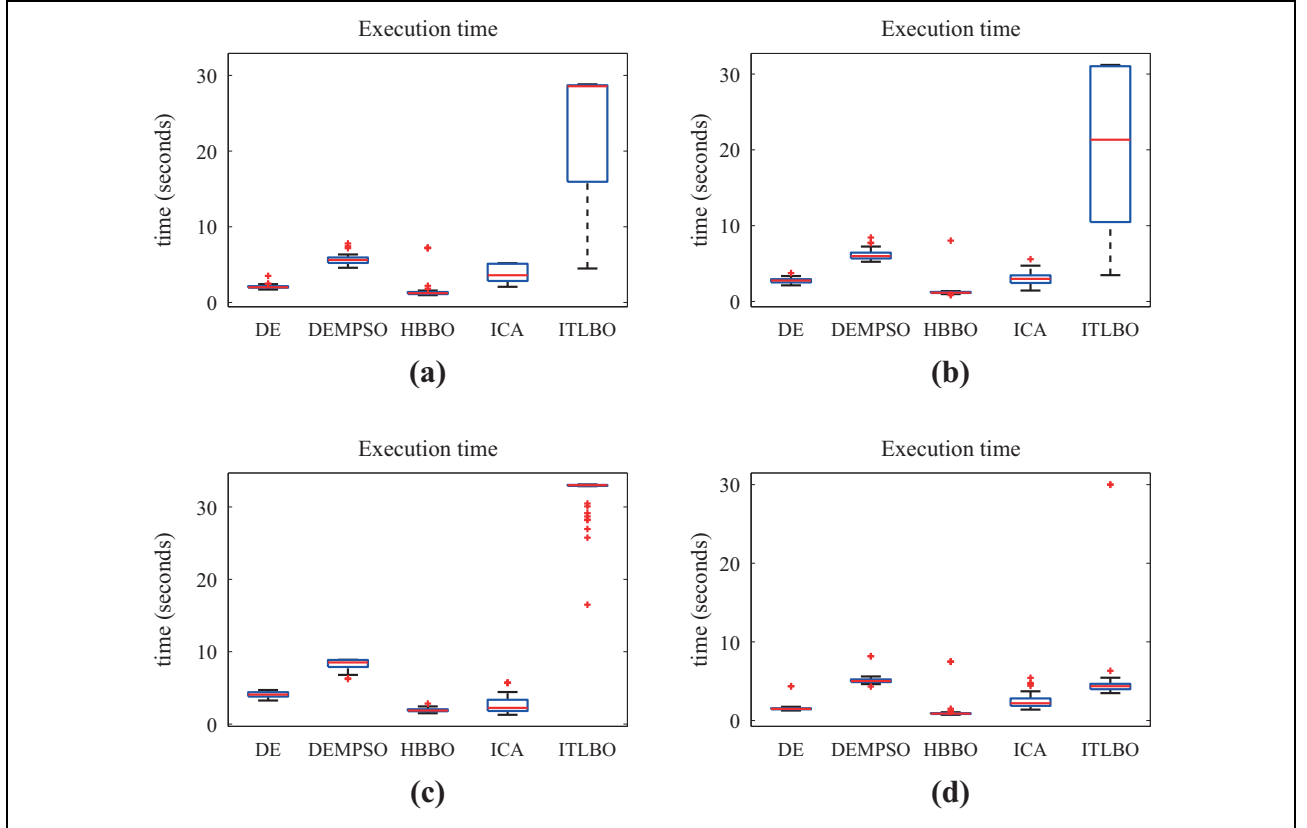


Figure 5. Execution time results for the considered meta-heuristic algorithms. (a) Youbot arms, (b) Stanford arms, (c) Baxter arms, and (d) hybrid arms.

manipulation tasks of dual-arm systems for offline applications. However, parallel computing has been used to reduce computational time of meta-heuristic algorithms.^{23,24} It would be interesting to implement those techniques to improve computational execution time of the proposed approach for online applications.

Non-parametric statistical test

Non-parametric statistical tests are usually used to compare a significant improvement among some compared algorithms.²⁵ The basic concepts of non-parametric procedure are the null hypothesis and the alternative hypothesis. The null hypothesis is assumed when the compared algorithms are equivalent. On the other hand, the alternative hypotheses represent significant difference among them. A level of significance α is used to determine at which level a hypothesis may be rejected. A p value provides information about whether a statistical hypothesis test is significant or not.

In this test, we proposed to use the sign test for pairwise comparison using the obtained results from the cooperative inverse kinematics test. We compared the DE against the DEMPSO, HBBO, ICA, and ITLBO algorithms. We also consider two levels of significance, $\alpha = 0.05$ and $\alpha = 0.1$. The sign test was performed using Matlab.

Table 6. Pairwise comparison of DE against the other meta-heuristics algorithms under sign test for position error results.

Robot	DE	DEMPSO	HBBO	ICA	ITLBO
Youbot	Wins (+)	31	33	44	41
	Loses (−)	19	17	6	9
	p Value	0.1189	0.0328	0.0000	0.0000
Stanford	Wins (+)	28	26	28	34
	Loses (−)	22	24	22	16
	p Value	0.4799	0.8877	0.4799	0.0153
Baxter	Wins (+)	37	25	33	43
	Loses (−)	13	25	17	7
	p Value	0.0009	1.0000	0.0328	0.0000
Hybrid	Wins (+)	18	21	37	24
	Loses (−)	32	29	13	26
	p Value	0.0649	0.3222	0.0009	0.8877

DE: differential evolution; MPSO: modified particle swarm optimization; HBBO: hybrid biogeography-based optimization; ICA: imperialist competitive algorithm; ITLBO: improved version of teaching-learning based optimization.

Table 6 shows the sign test results for the position errors of the cooperative inverse kinematics test. In the case of the Youbot arms, DE outperformed HBBO, ICA, and ITLBO with a level of significance below to 0.05, and DE performed similar to DEMPSO. In the Stanford arms, DE shows a significant improvement over the ITLBO

Table 7. Pairwise comparison of DE against the other meta-heuristics algorithms under sign test for displacement error results.

Robot	DE	DEMPSO	HBBO	ICA	ITLBO
Youbot	Wins (+)	50	45	47	39
	Loses (−)	0	5	3	11
	p Value	0.0000	0.0000	0.0000	0.0001
Stanford	Wins (+)	50	50	50	50
	Loses (−)	0	0	0	0
	p Value	0.0000	0.0000	0.0000	0.0000
Baxter	Wins (+)	50	50	50	49
	Loses (−)	0	0	0	1
	p Value	0.0000	0.0000	0.0000	0.0000
Hybrid	Wins (+)	33	20	11	23
	Loses (−)	17	30	39	27
	p Value	0.0328	0.2026	0.0001	0.6718

DE: differential evolution; MPSO: modified particle swarm optimization; HBBO: hybrid biogeography-based optimization; ICA: imperialist competitive algorithm; ITLBO: improved version of teaching-learning based optimization.

algorithms. For the Baxter arms, DE performed better than DEMPSO, ICA, and ITLBO with a level of significance below to 0.05, and the performance of DE is similar to HBBO. Finally, in the Hybrid system, DE outperformed ICA algorithms with a level of significance below to 0.05 and DEMPSO outperformed DE with a level of significance below to 0.1.

Table 7 presents the sign test results for the displacement errors of the cooperative inverse kinematics test. As we can see, the DE algorithm outperformed the others with a level of significance below to 0.05 in the Youbot, Stanford, and the Baxter arms. In the case of the hybrid system, DE shows a significant improvement over DEMPSO and ICA algorithms with a level of significance below to 0.05.

Table 8 reports the sign test results for the execution time of the cooperative inverse kinematics test. Here, we notice that HBBO reported a significant difference against DE with a level of significance below to 0.05 in all considered dual-arm systems. Moreover, DE shows a detected difference against DEMPSO, ICA, and ITLBO with a level of significance below to 0.05 for all dual-arm systems, except to ICA in the Stanford arms. In this case, DE shows an improvement over ICA with a level of significance below to 0.1.

Based on the results of the sign tests, DE performed better than the other algorithms with respect to position and displacement results in general. However, HBBO prove to be superior than DE with respect to execution time results. To conclude, we highly recommend the use of DE to solve the cooperative inverse kinematics problems; however, any other meta-heuristic algorithm can also be used to solve these problems. For example, it would be interesting to use HBBO to solve the problem, because it had a good performance as well.

Table 8. Pairwise comparison of DE against the other meta-heuristics algorithms under sign test for execution time results.

Robot	DE	DEMPSO	HBBO	ICA	ITLBO
Youbot	Wins (+)	50	8	49	50
	Loses (−)	0	42	1	0
	p Value	0.0000	0.0000	0.0000	0.0000
Stanford	Wins (+)	50	1	32	50
	Loses (−)	0	49	18	0
	p Value	0.0000	0.0000	0.0649	0.0000
Baxter	Wins (+)	50	0	8	50
	Loses (−)	0	50	42	0
	p Value	0.0000	0.0000	0.0000	0.0000
Hybrid	Wins (+)	50	4	47	50
	Loses (−)	0	46	3	0
	p Value	0.0000	0.0000	0.0000	0.0000

DE: differential evolution; MPSO: modified particle swarm optimization; HBBO: hybrid biogeography-based optimization; ICA: imperialist competitive algorithm; ITLBO: improved version of teaching-learning based optimization.

Cooperative path tracking tests

The purpose of the test is to compare the proposed approach against a conventional method under some cooperative path tracking tasks. In this test, two different path trajectories with different initial joint configurations were selected to compare the performance of the proposed and conventional methods against singularities. The initial joints configurations were intentionally chosen to make the problem difficult to solve. For this test, a circular and a sinusoidal trajectories have been chosen, these trajectories are a benchmark in these area.

In the proposed approach, we used the DE algorithm to solve the cooperative path tracking tasks and its parameter settings were selected similar to the cooperative inverse kinematics test. In the conventional method, we set the highest priority task to the relative motion between the two end-effectors. The secondly task was set to the desired end-effector position with respect to the main manipulator. The gain for the priority task K_r was set to 0.3 and the gain for the secondly task was set to 0.7. To avoid singularities in the conventional method, we proposed to use the Moore–Penrose pseudoinverse of the Jacobian matrix based on singular value decomposition.

In this test, we proposed to use the mentioned dual-arm KUKA Youbot system and dual-arm Baxter system from cooperative inverse kinematics test. We also used the same ${}^w\mathbf{T}_{a_o}$, ${}^w\mathbf{T}_{b_o}$, $\mathbf{p}_r$, $\mathbf{q}_l$, and $\mathbf{q}_u$ configurations. The circle trajectory $\mathbf{p}_k^* = [x_k \ y_k \ z_k]^T$ for the Youbot system is given by the following equations

$$\begin{aligned} x_k &= 0.4 \\ y_k &= 0.05 \cos(\theta_k) \\ z_k &= 0.35 + 0.05 \sin(\theta_k) \end{aligned}$$

where $k \in [1, M]$ with $M = 50$ points and $\theta \in [0, 2\pi]$. The sinusoidal trajectory $\mathbf{p}_k^*$ for the Baxter system is given by

$$x_k = 0.9$$

$$z_k = 0.8 + 0.05 \sin(30y_k)$$

where $k \in [1, M]$ with $M = 50$ points and $y_k \in [0.0, 0.4]$.

For the first part of the test, we set the initial joint configuration for the KUKA Youbot arms as

$$\mathbf{q}_{\text{old}} = \begin{bmatrix} 0 & 1.5708 & -1.0472 & 0 & 0 \\ 0 & 1.5708 & -1.0472 & 0 & 0 \end{bmatrix}^T \quad (12)$$

and the initial joint configuration for the Baxter arms was selected as

$$\mathbf{q}_{\text{old}} = \begin{bmatrix} -0.7071 & 0 & 0 & 0 & 0 & 0 & 0 \\ -0.7071 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^T \quad (13)$$

The cooperative path tracking results for these initial joint configurations are given in the next paragraphs.

Figure 6 shows the path tracking and the position error results for the dual-arm Baxter system on the sinusoidal trajectory using the initial joint configuration in equation (13). The Figure 6(a) and (b) shows the results for the tracking of manipulators A and B , respectively. In these cases, the reported results show that the proposed approach $\mathbf{p}_{a_e}^1$ and the conventional method $\mathbf{p}_{a_e}^2$ perfectly follow the desired trajectory $\mathbf{p}_{a_e}^*$. The results $\mathbf{p}_{b_e}^1$ of the proposed approach and $\mathbf{p}_{b_e}^2$ of the conventional method report that they follow correctly the manipulator B . The position error results for manipulators A and B are given in Figure 6(c) and (d), respectively. Here, we can observe that the proposed approach and the conventional method performed below to an error of 0.001.

The joint displacement results for Baxter arms on the sinusoidal trajectory using the initial joint configuration in equation (13) are given in Figure 7. Figure 7(a) and (b) presents the joint displacement results for the proposed approach of manipulators A and B , respectively. It is shown that the proposed approach reported smooth joint displacement in both manipulators. Then, the joint displacement results for the conventional method of manipulators A and B are reported in Figure 7(c) and (d), respectively. Here, we can observe that the conventional method reported smooth joint displacement in both manipulators as well.

Figure 8 reports the path tracking and the position error results for the dual-arm KUKA Youbot system on the circle trajectory using the initial joint configuration in equation (12). The tracking results for manipulator A of the proposed approach $\mathbf{p}_{a_e}^1$ matches with the desired trajectory $\mathbf{p}_{a_e}^*$, as well as the conventional method $\mathbf{p}_{a_e}^2$, see Figure 8(a). Similar tracking results for manipulator B are reported in Figure 8(b), where the results for the proposed approach and the conventional method follow correctly the manipulator A . Furthermore, the position error results for manipulators A and B are presented in Figure 8(c) and (d), respectively.

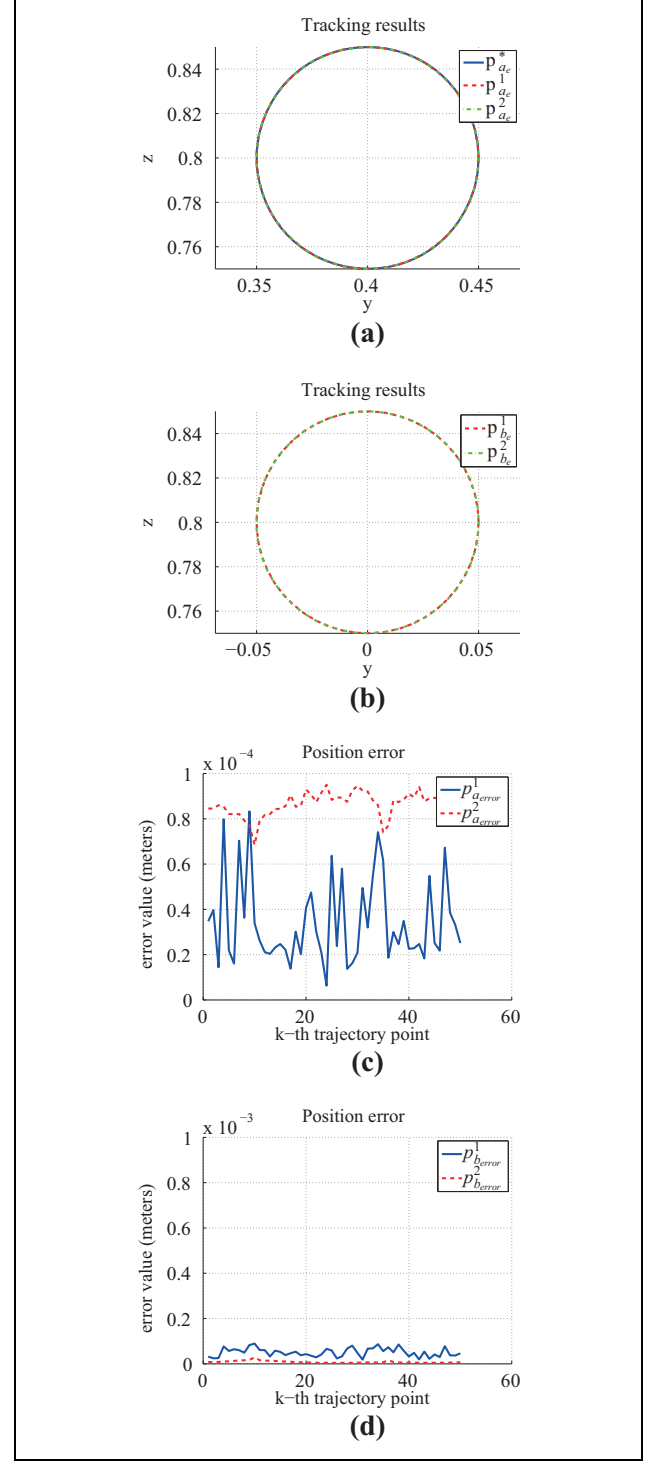


Figure 6. Tracking and position error results for Baxter arms on sinusoidal trajectory using the initial joint configuration in equation (13). (a) and (b) show the tracking results, where $(\mathbf{p}_{a_e}^1, \mathbf{p}_{b_e}^1)$ and $(\mathbf{p}_{a_e}^2, \mathbf{p}_{b_e}^2)$ are the results for the proposed approach and the conventional method, respectively. The desired trajectory is given by $\mathbf{p}_{a_e}^*$. (c) and (d) present the position error results, where $(p_{a_{\text{error}}}^1, p_{b_{\text{error}}}^1)$ and $(p_{a_{\text{error}}}^2, p_{b_{\text{error}}}^2)$ are the results for the proposed approach and the conventional method, respectively.

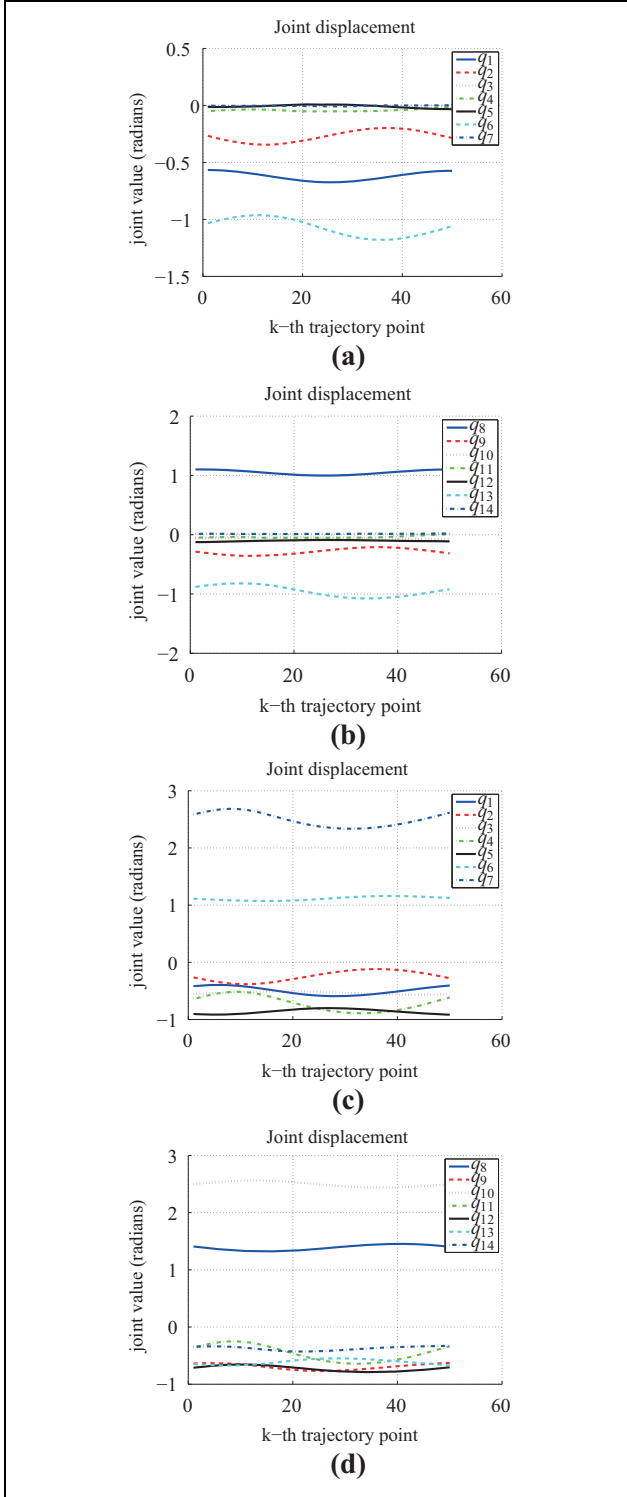


Figure 7. Joint displacement results for Baxter arms on sinusoidal trajectory using the initial joint configuration in equation (13). (a) and (b) show the results for the proposed approach. (c) and (d) present the results for the conventional method.

In this case, the proposed approach presented an error below to 0.01 mm at the first point in the trajectory. For the other points in the trajectory, the proposed approach

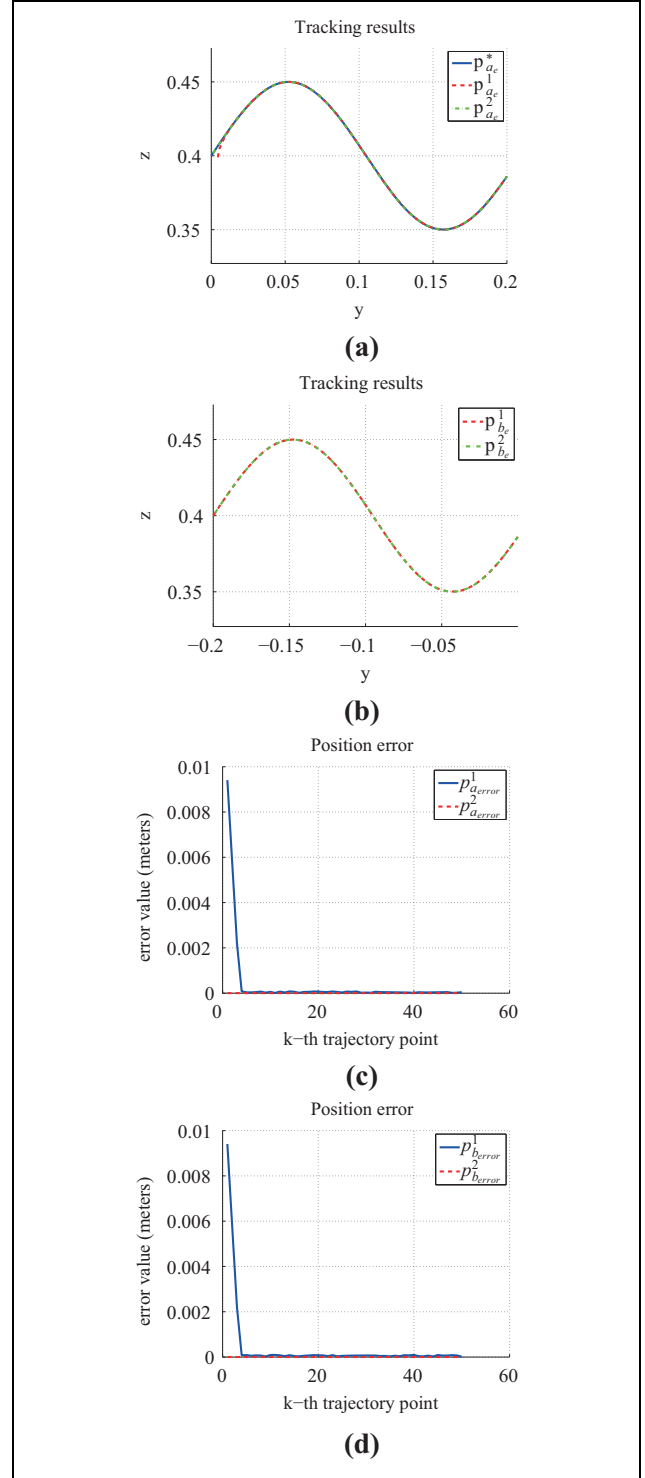


Figure 8. Tracking and position error results for KUKA Youbot arms on sinusoidal trajectory using the initial joint configuration (equation (12)). (a) and (b) show the tracking results, where $(\mathbf{p}_{a_e}^1, \mathbf{p}_{b_e}^1)$ and $(\mathbf{p}_{a_e}^2, \mathbf{p}_{b_e}^2)$ are the results for the proposed approach and the conventional method, respectively. The desired trajectory is given by $\mathbf{p}_e^*$. (c) and (d) present the position error results, where $(\mathbf{p}_{a_{error}}^1, \mathbf{p}_{b_{error}}^1)$ and $(\mathbf{p}_{a_{error}}^2, \mathbf{p}_{b_{error}}^2)$ are the results for the proposed approach and the conventional method, respectively.

and the conventional method reported error results below to 0.001 mm.

The joint displacement results for KUKA Youbot arms on the circle trajectory using the initial joint configuration in equation (12) are presented in Figure 9. The proposed approach shows a smooth joint displacement during the tracking task in manipulators *A* and *B*, see Figure 9(a) and (b), respectively. Similarly, the conventional method reported a smooth joint displacement in the trajectory, as it is shown in Figure 9(c) and (d).

For the second part of this test, the initial joint configuration for the KUKA Youbot arms was set to

$$\mathbf{q}_{\text{old}} = \begin{bmatrix} 0 & 1.5708 & 0 & 1.5708 & 0 \\ 0 & 1.5708 & 0 & 1.5708 & 0 \end{bmatrix}^T \quad (14)$$

and the initial joint configuration for the Baxter arms was selected as

$$\mathbf{q}_{\text{old}} = \begin{bmatrix} 0 & -1.5708 & -1.5708 & 0 & 0 & 0 & 0 \\ 0 & -1.5708 & -1.5708 & 0 & 0 & 0 & 0 \end{bmatrix}^T \quad (15)$$

The cooperative path tracking results for these initial joint configurations are given below.

Figure 10 shows the path tracking and the position error results for the dual-arm Baxter system on the sinusoidal trajectory using the initial joint configuration in equation (15). The Figure 10(a) and (b) shows the results for the tracking of manipulators *A* and *B*, respectively. In these cases, the reported results show that the proposed approach $\mathbf{p}_{a_e}^1$ perfectly follows the desired trajectory $\mathbf{p}_{a_e}^*$ and the result $\mathbf{p}_{b_e}^1$ reports that it follows perfectly the manipulator *B*, in contrast, the results of the conventional method $\mathbf{p}_{a_e}^2$ reported bigger error results at the beginning of the given task. The position error results for manipulators *A* and *B* are given in Figure 10(c) and 10(d), respectively. Here, we can observe that the proposed approach performed below to an error of 0.001, on the other hand, the conventional method reported bigger results at the beginning of the path.

The joint displacement results for Baxter arms on the sinusoidal trajectory using the initial joint configuration in equation (15) are given in Figure 11. Figure 11(a) and (b) present the joint displacement results for the proposed approach of manipulators *A* and *B*, respectively. It is shown that the proposed approach reported smooth joint displacement in both manipulators. Then, the joint displacement results for the conventional method of manipulators *A* and *B* are reported in Figure 11(c) and (d), respectively. Here, we can observe abrupt changes in the joint displacement values presented at the beginning of the task.

Figure 12 reports the path tracking and the position error results for the dual-arm KUKA Youbot system on the circle trajectory using the initial joint configuration in equation (14). The tracking results for manipulator *A* of the proposed approach $\mathbf{p}_{a_e}^1$ perfectly match with the desired trajectory

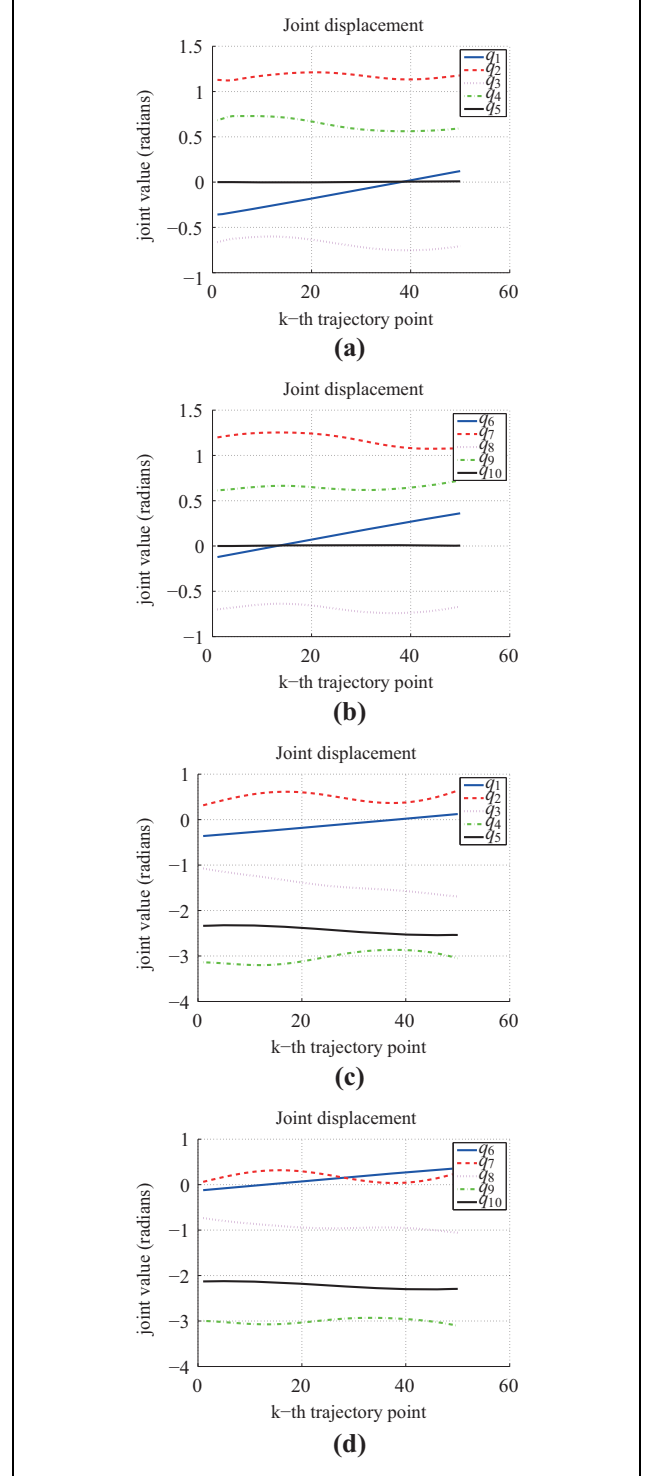


Figure 9. Joint displacement results for KUKA Youbot arms on sinusoidal trajectory using the initial joint configuration (equation (12)). (a) and (b) show the results for the proposed approach. (c) and (d) present the results for the conventional method.

$\mathbf{p}_{a_e}^*$; on the contrary, the conventional method does not follow the trajectory correctly, see Figure 12(a). Similar tracking results for manipulator *B* are reported in Figure 12(b), where the results for the proposed approach for

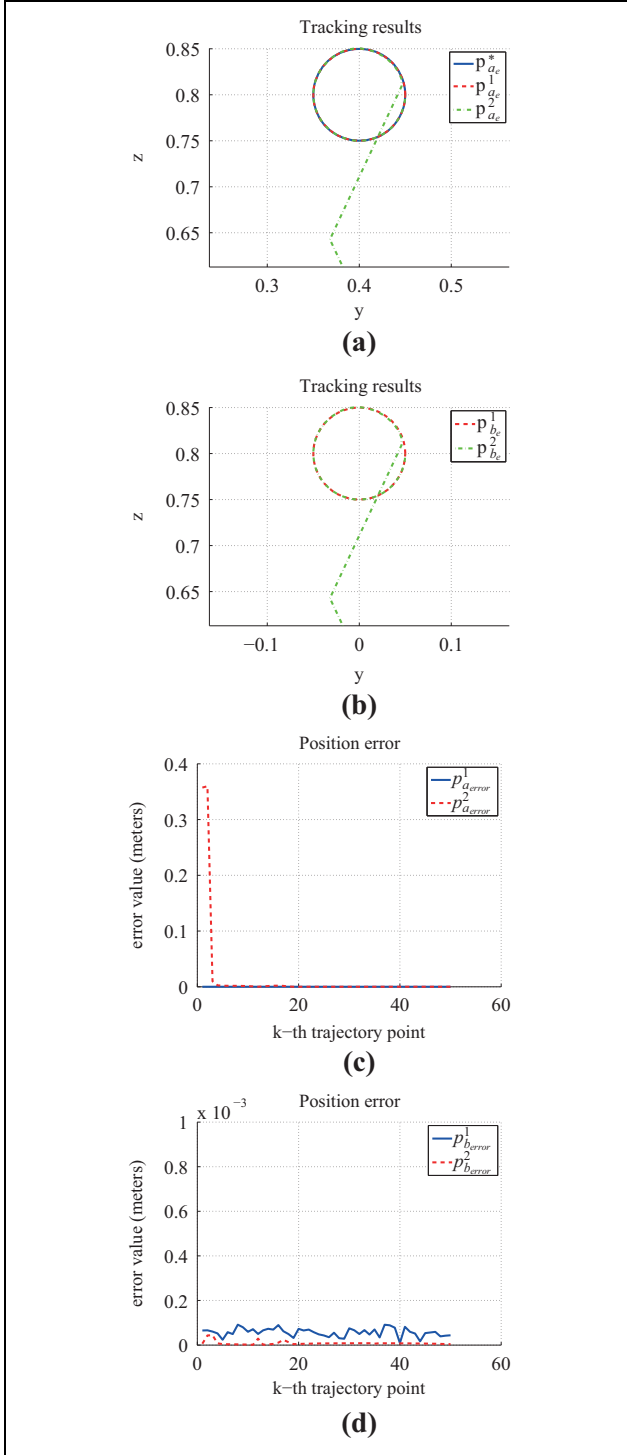


Figure 10. Tracking and position error results for Baxter arms on sinusoidal trajectory using the initial joint configuration (equation (15)). (a) and (b) show the tracking results, where $(p_{a_e}^1, p_{b_e}^1)$ and $(p_{a_e}^2, p_{b_e}^2)$ are the results for the proposed approach and the conventional method, respectively. The desired trajectory is given by $p_{a_e}^*$. (c) and (d) present the position error results, where $(p_{a_{error}}^1, p_{b_{error}}^1)$ and $(p_{a_{error}}^2, p_{b_{error}}^2)$ are the results for the proposed approach and the conventional method, respectively.

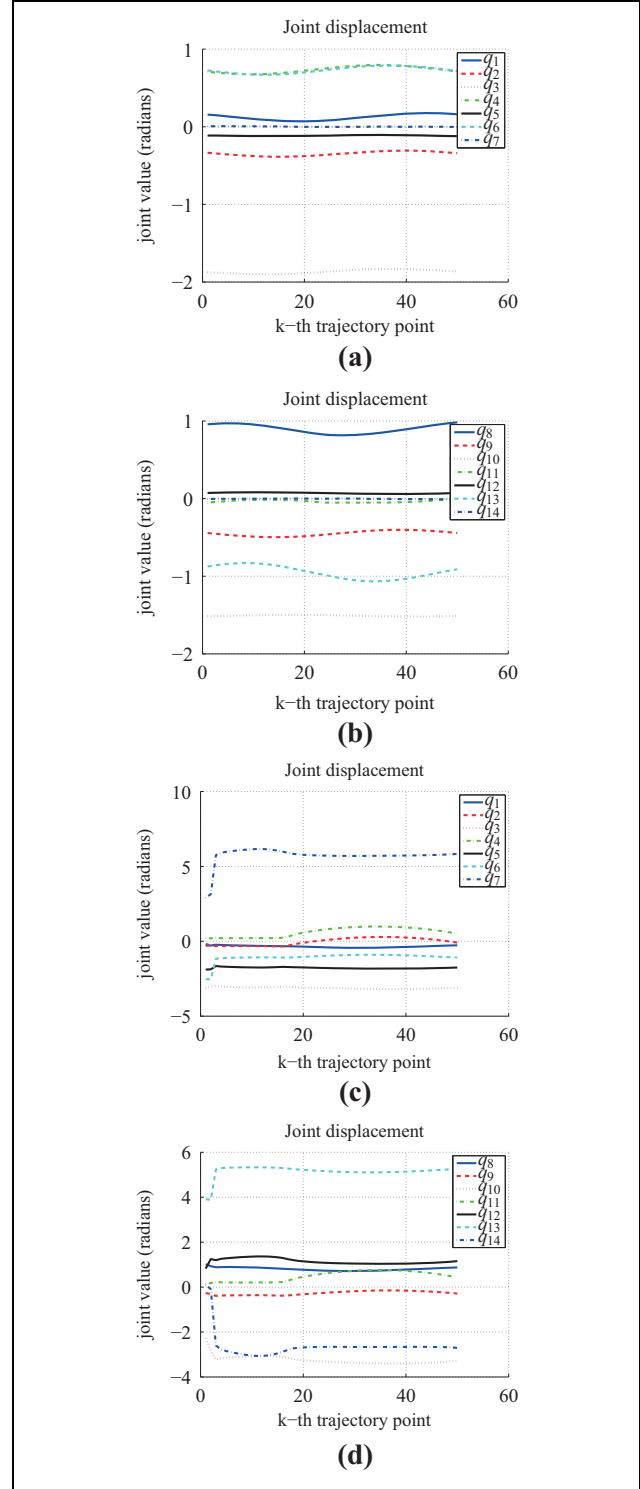


Figure 11. Joint displacement results for Baxter arms on sinusoidal trajectory using the initial joint configuration (equation (15)). (a) and (b) show the results for the proposed approach. (c) and (d) present the results for the conventional method.

manipulator *B* perfectly follow the manipulator *A*; in contrast, the conventional method reported that it follows the manipulator *A* with the same error tracking. Furthermore,

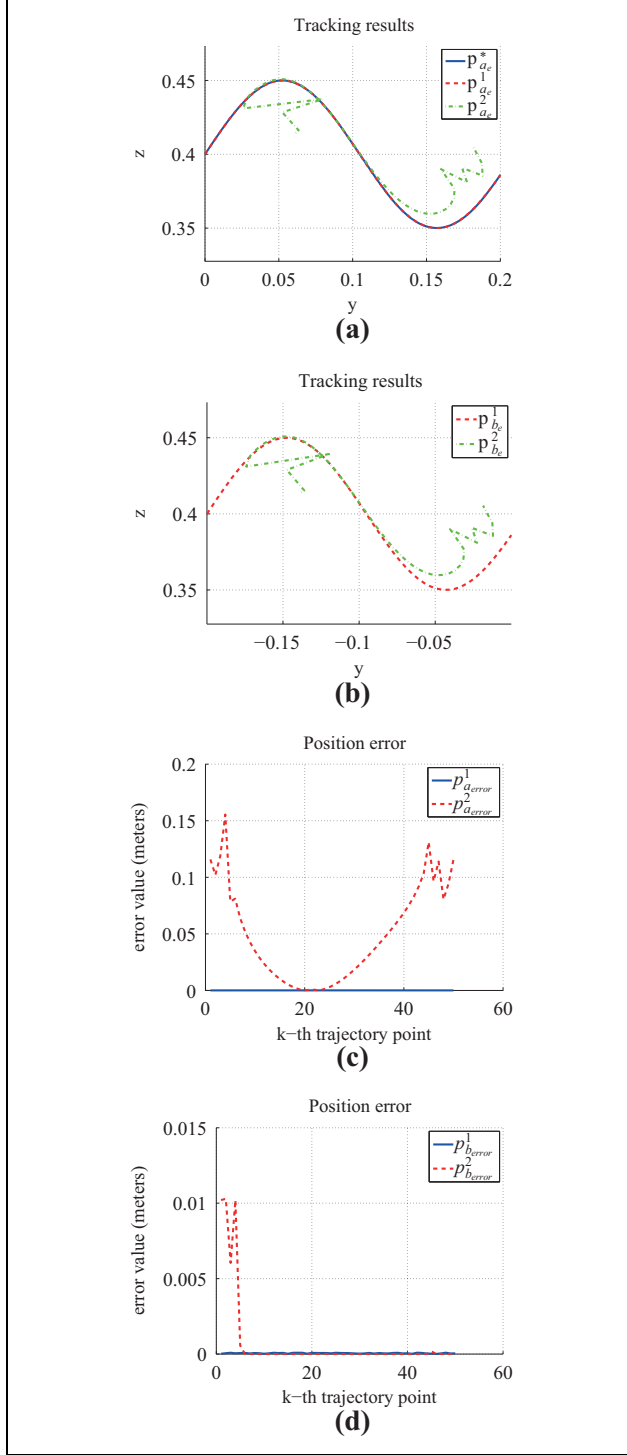


Figure 12. Tracking and position error results for KUKA Youbot arms on sinusoidal trajectory using the initial joint configuration (equation (14)). (a) and (b) show the tracking results, where $(\mathbf{p}_{a_e}^1, \mathbf{p}_{b_e}^1)$ and $(\mathbf{p}_{a_e}^2, \mathbf{p}_{b_e}^2)$ are the results for the proposed approach and the conventional method, respectively. The desired trajectory is given by $\mathbf{p}_{a_e}^*$. (c) and (d) present the position error results, where $(\mathbf{p}_{a_{error}}^1, \mathbf{p}_{b_{error}}^1)$ and $(\mathbf{p}_{a_{error}}^2, \mathbf{p}_{b_{error}}^2)$ are the results for the proposed approach and the conventional method, respectively.

the position error results for manipulators *A* and *B* are presented in Figure 12(c) and (d), respectively. In this case, the proposed approach performed below an error of 0.001 mm; in contrast, the conventional method reported bigger results in both manipulators.

The joint displacement results for KUKA Youbot arms on the circle trajectory using the initial joint configuration in equation (14) are presented in Figure 13. The proposed approach shows a smooth joint displacement during the tracking task in manipulators *A* and *B*, see Figure 13(a) and (b), respectively. In contrast, the conventional method reported abrupt changes at some particular point in the trajectory, as it is shown in Figure 13(c) and (d).

Accordingly, with the cooperative path tracking results, the conventional method suffers from singularities. As we can observe in some points of the trajectory, the inverse kinematics task in the conventional method fails. This problem occurs because it requires the inversion of a Jacobian matrix that has become rank deficient in a particular joint configuration. Therefore, the trajectory tracking has an abrupt change in the joint displacement. In contrast, the proposed approach proved to be able to avoid such joint configuration problems, as it does not require the inversion of any Jacobian matrix.

Experimental results

Real experiments were performed to show the applicability of the proposed approach. In this test, we performed two cooperative coordinate manipulation tasks for a dual-arm system. The cooperative tasks are a circle and a sinusoidal trajectory tracking. We considered to use the dual-arm KUKA Youbot system illustrated in Figure 14. To solve the constrained optimization problem, we used the DE algorithm with the standard DE/rand/1/bin mutation strategy. We selected the standard DE algorithm based on the results given by simulations experiments. In addition, we set the amplification factor $F = 0.6$, the crossover constant $C_R = 0.9$, $N = 150$ individuals, and a total of 500 iterations. The allowed tolerance was set to $s_{tol} = 0.0001$.

The DH parameters for manipulators *A* and *B* are the same, and it is given in Table 4. The initial joint configurations were selected as

$$\mathbf{q}_{old} = [0 \quad \pi/2 \quad 0 \quad \pi/2 \quad 0 \quad 0 \quad \pi/2 \quad 0 \quad \pi/2 \quad 0]^T$$

and the lower $\mathbf{q}_l = [\mathbf{q}_{a_l} \quad \mathbf{q}_{b_l}]^T$ and upper $\mathbf{q}_u = [\mathbf{q}_{a_u} \quad \mathbf{q}_{b_u}]^T$ joint limits were given as

$$\begin{aligned} \mathbf{q}_{a_l} = \mathbf{q}_{b_l} &= [-2.9496 \quad -1.1345 \quad -2.6180 \\ &\quad -1.7837 \quad -2.9234]^T \\ \mathbf{q}_{a_u} = \mathbf{q}_{b_u} &= [2.9496 \quad 1.5708 \quad 2.5482 \\ &\quad 1.7890 \quad 2.9234]^T \end{aligned}$$

The transformations ${}^w\mathbf{T}_{a_o}$ and ${}^w\mathbf{T}_{b_o}$ were fixed as

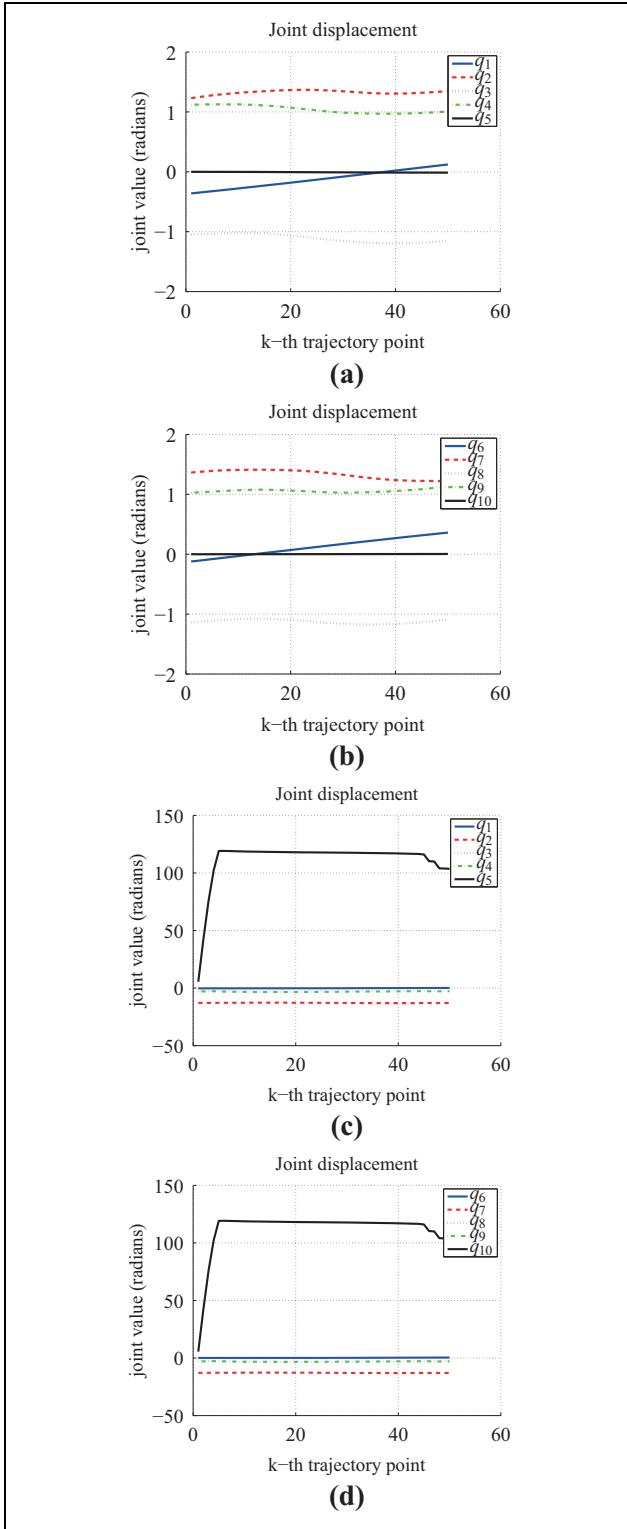


Figure 13. Joint displacement results for KUKA Youbot arms on sinusoidal trajectory using the initial joint configuration (equation (14)). (a) and (b) show the results for the proposed approach. (c) and (d) present the results for the conventional method.



Figure 14. Dual-arm KUKA Youbot system. It is composed of two manipulators with 5-DOF each arm. DOF: degree of freedom.

$${}^w\mathbf{T}_{a_o} = \begin{bmatrix} 1 & 0 & 0 & 0.0 \\ 0 & 1 & 0 & 0.95 \\ 0 & 0 & 1 & 0.140 \\ 0 & 0 & 0 & 1 \end{bmatrix}, {}^w\mathbf{T}_{b_o} = \begin{bmatrix} 1 & 0 & 0 & 0.0 \\ 0 & 1 & 0 & -0.95 \\ 0 & 0 & 1 & 0.140 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Finally, the relative position $\mathbf{p}_r$ between the end-effector of manipulators A and B was selected as $\mathbf{p}_r = [0.0 \quad -(0.95 + 0.95) \quad 0.0]^T$.

Simulation results show the effectiveness of the proposed approach for solving a given cooperative path tracking task. However, no control laws were required in the simulation experiments. The real experiments were performed using C++ and Robot Operating System (ROS), these experiments require a controller to track the path computed by the proposed approach. In the experiments, an existing ROS component was used to access the KUKA Youbot hardware and also the integrated manipulator proportional–integral–derivative (PID) controller to perform the task. It is needed to add an offset $\mathbf{q}_{\text{offset}}$ to use the integrated manipulator controller. This offset is defined as

$$\mathbf{q}_{\text{offset}} = [2.9496 \quad 2.7053 \quad -2.5482 \quad 3.3598 \quad 2.9234]^T$$

then, we compute the joints configuration $\mathbf{q}_{a_{\text{ctrl}}}$ and $\mathbf{q}_{b_{\text{ctrl}}}$ needed for the manipulators A and B controller, respectively, as follows

$$\mathbf{q}_{a_{\text{ctrl}}} = -\mathbf{q}_a + \mathbf{q}_{\text{offset}}$$

$$\mathbf{q}_{b_{\text{ctrl}}} = -\mathbf{q}_b + \mathbf{q}_{\text{offset}}$$

The circle trajectory $\mathbf{p}_k^* = [x_k \quad y_k \quad z_k]^T$ is given by the following equations

$$x_k = 0.35$$

$$y_k = 0.95 + 0.05 \cos(\theta_k)$$

$$z_k = 0.50 + 0.05 \sin(\theta_k)$$

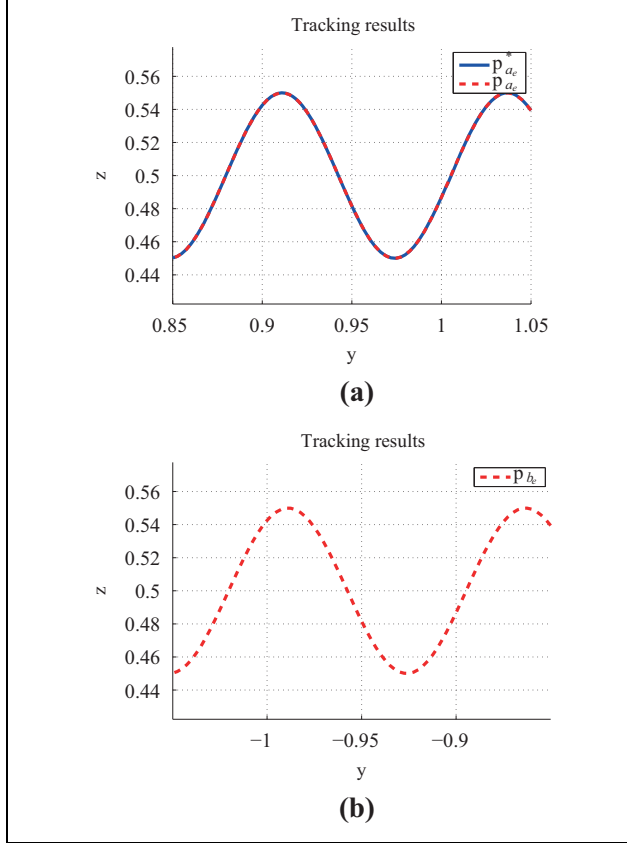


Figure 15. Tracking results for KUKA Youbot arms on sinusoidal trajectory. Values p_{ae} and p_{be} are (a) the manipulator A and (b) the manipulator B end-effector position, respectively. The desired trajectory is given by p_{ae}^* .

where $k \in [1, M]$ with $M = 80$ points and $\theta \in [0, 2\pi]$. The sinusoidal trajectory p_k^* is given by

$$\begin{aligned} x_k &= 0.35 \\ z_k &= 0.50 + 0.05 \sin(50y_k) \end{aligned}$$

where $k \in [1, M]$ with $M = 120$ points and $y_k \in [-0.85, 1.05]$.

The trajectory results for sinusoidal tracking test are presented in Figure 15. As we can see in Figure 15(a), the position of the end-effector p_{ae} for manipulator A perfectly matched the desired position p_{ae}^* . It is also shown that the end-effector position p_{be} for manipulator B follows successfully the manipulator A, see Figure 15(b). Then, Figure 16 shows the errors $p_{a\text{error}}$ and $p_{b\text{error}}$ for the end-effector of manipulators A and B, respectively. Here, we notice that the proposed approach performed below to an error of 0.0002 mm, for each point in the path. Finally, in Figure 17, the results for the joint displacement are shown. Here, the reported results illustrate a smooth trajectory for the given path.

The path tracking results of the circle path are presented in Figure 18. It is shown in Figure 18(a) that the position of the end-effector p_{ae} matches to the position p_{ae}^* for all

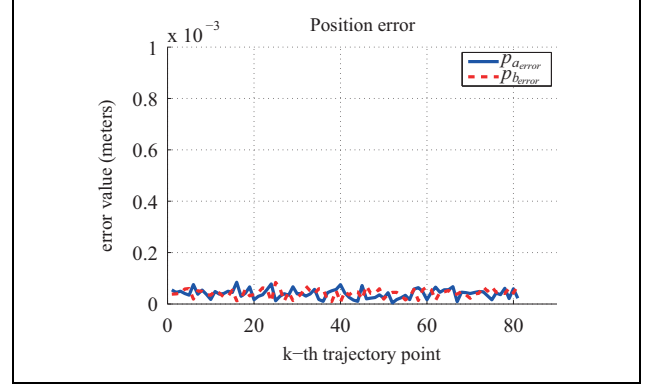


Figure 16. Position error results for KUKA Youbot arms on sinusoidal trajectory. Values $p_{a\text{error}}$ and $p_{b\text{error}}$ are the manipulator A and the manipulator B end-effector errors, respectively.

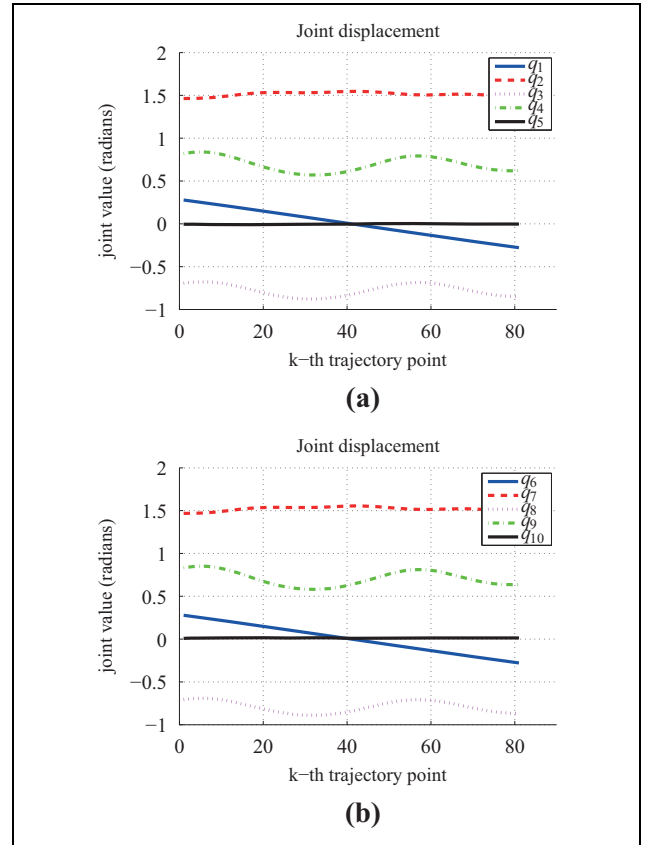


Figure 17. Joint displacement results for KUKA Youbot arms on sinusoidal trajectory. (a) Manipulator A and (b) manipulator B.

points in the path. In Figure 18(b), the position of the end-effector p_{be} follows correctly the manipulator A. Then, the errors results of manipulators' positions A and B are given in Figure 19. Similarly to the sinusoidal case, the reported error performed below to 0.0002 mm of error. At last, Figure 20 shows smooth results for the joints displacement of the dual-arm system.

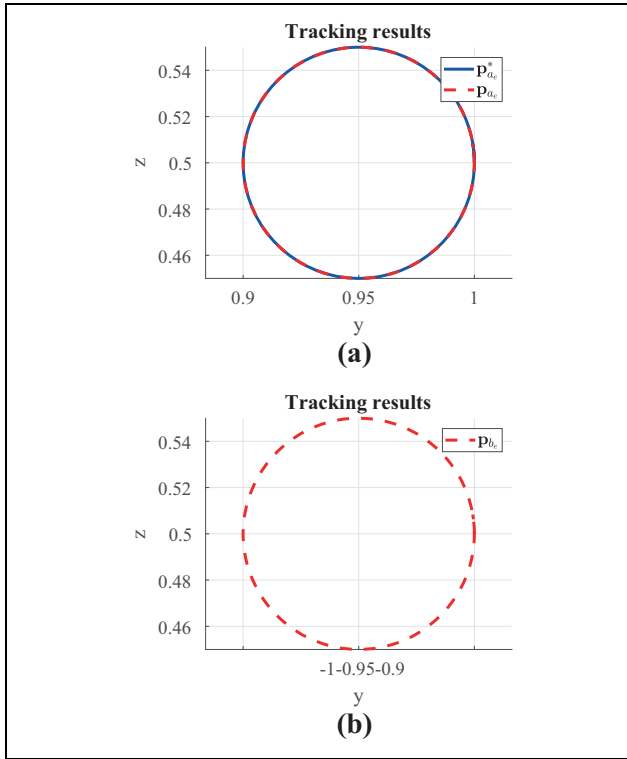


Figure 18. Tracking results for KUKA Youbot arms on circle trajectory. Values p_{a*} and p_{b*} are (a) the manipulator A and (b) the manipulator B end-effector position, respectively. The desired trajectory is given by p_{a*} .

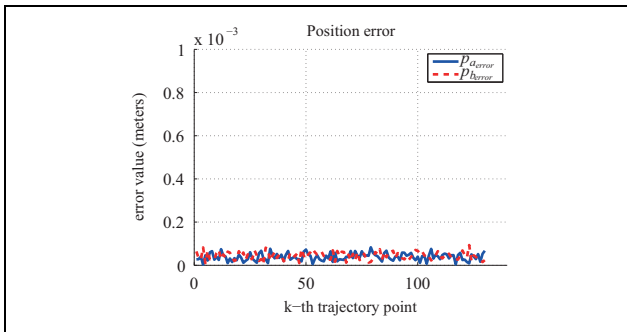


Figure 19. Position error results for KUKA Youbot arms on circle trajectory. Values p_{a_error} and p_{b_error} are the manipulator A and the manipulator B end-effector errors, respectively.

Based on the results of real experiments, we can conclude that the proposed approach solved successfully the cooperative dual-arm manipulation for the given path tracking tasks.

Conclusions

In this work, we introduced an approach for solving the coordinate dual-arm cooperative manipulation tasks based

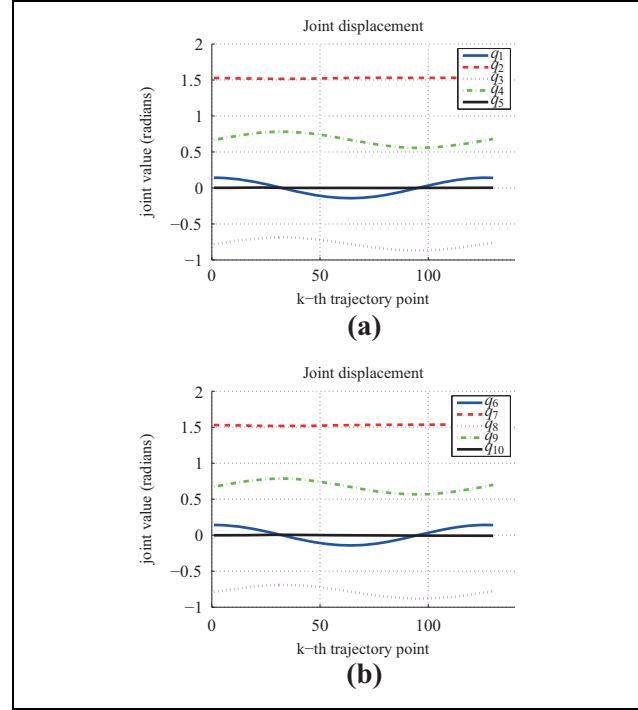


Figure 20. Joint displacement results for KUKA Youbot arms on circle trajectory.

on DE. The proposed method represents the kinematics structures of dual-arm systems using the DH convention. Simulation and real experiments were performed to prove the effectiveness of the proposed approach. The simulation included a comparison among some selected meta-heuristic algorithms such as DE, DEMPSO, HBBO, ICA, and ITLBO. In addition, different dual-arm system composed of different manipulators were considered in simulations to test these algorithm under a cooperative inverse kinematics test. In this test, DE and HBBO performed better than the others with the smaller execution time and position error results. They also presented the smaller data distribution. However, it was not reported a significant difference between DE and HBBO. Then, the sign test was performed to detect a significant improvement of DE against the others meta-heuristic algorithms. In this test, DE proved to be superior to other algorithms with respect to position and displacement results. In contrast, HBBO overcomes the other algorithms with respect to execution time results. Moreover, a comparison was performed to compare the proposed approach against a conventional method under some cooperative path tracking tasks. In this test, the proposed approach proved to be able to solve the coordination tasks under singularities conditions. In contrast, the conventional method solves the same coordination tasks poorly. In real experiments, a dual-arm KUKA Youbot system mobile manipulator was considered as well. In this case, the proposed approach solves the cooperative tasks successfully, where the path

tracking results perfectly match the given path tracking tasks. In conclusion, the proposed approach can solve the dual-arm cooperative manipulation for coordinate manipulations tasks. This approach can avoid singularities, because the inversion of Jacobian matrices are not required. Moreover, it is able to solve cooperative tasks for different dual-arm systems structures, including revolute and prismatic joints. Finally, the proposed method is able to handle joint limits constraints and it is suitable for redundant and non-redundant manipulators structures. This work can be extended to solve problems of coordinate cooperation systems composed of n number of manipulators, which is an interesting topic for future research.

Acknowledgement

The authors thank the support of CONACYT México and the Universidad de Guadalajara.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: CONACYT projects CB256769 and CB258068 (project supported by Fondo de Investigación Sectorial para la Educación FOINS 241246).

ORCID iD

Jesús Hernández-Barragán  <https://orcid.org/0000-0001-7518-1668>

Alma Y Alanis  <https://orcid.org/0000-0001-9600-779X>

References

- Ortenzi D, Muthusamy R, Freddi A, et al. Dual-arm cooperative manipulation under joint limit constraints. *Robot Auton Syst* 2018; 99: 110–120.
- Smith C, Karayiannidis Y, Nalpantidis L, et al. Dual arm manipulation: a survey. *Robot Auton Syst* 2012; 60(10): 1340–1353.
- Lewis CL. Trajectory generation for two robots cooperating to perform a task. In: *Proceedings of IEEE international conference on robotics and automation*, Minneapolis, USA, 22–28 April 1996, vol 2. pp. 1626–1631. IEEE. DOI: 10.1109/ROBOT.1996.506945.
- Jamisola RS and Roberts RG. A more compact expression of relative Jacobian based on individual manipulator Jacobians. *Robot Auton Syst* 2015; 63: 158–164.
- Freddi A, Longhi S, Monteri A, et al. Redundancy analysis of cooperative dual-arm manipulators. *Int J Adv Robot Syst* 2016; 13(5): 1729881416657754.
- Hu Y, Huang B, and Yang GZ. Task-priority redundancy resolution for co-operative control under task conflicts and joint constraints. In: *2015 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. Hamburg, Germany, 28 September–2 October 2015, pp. 2398–2405. IEEE. DOI: 10.1109/IROS.2015.7353702.
- Spong MW and Vidyasagar M. *Robot dynamics and control*. Hoboken: John Wiley & Sons, 2008.
- Liu T, Lei Y, Han L, et al. Coordinated resolved motion control of dual-arm manipulators with closed chain. *Int J Adv Robot Syst* 2016; 13(3): 80.
- Selvaraj C, Siva Kumar R, and Karnan M. A survey on application of bio-inspired algorithms. *Int J Comput Sci Inform Technol* 2014; 5(1): 366–370.
- Huang HC, Chen CP, and Wang PR. Particle swarm optimization for solving the inverse kinematics of 7-DOF robotic manipulators. In: *2012 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, Seoul, South Korea, 14–17 October 2012, pp. 3105–3110. IEEE.
- Nearchou AC. Solving the inverse kinematics problem of redundant robots operating in complex environments via a modified genetic algorithm. *Mech Mach Theory* 1998; 33(3): 273–292.
- Yang Y, Peng G, Wang Y, et al. A new solution for inverse kinematics of 7-DOF manipulator based on genetic algorithm. In: *2007 IEEE International Conference on Automation and Logistics*. pp. 1947–1951.
- Pham DT, Castellani M, and Fahmy AA. Learning the inverse kinematics of a robot manipulator using the bees algorithm. In: *6th IEEE international conference on industrial informatics, 2008 INDIN 2008*, Daejeon, South Korea, 13–16 July 2008, pp. 493–498. IEEE. DOI: 10.1109/INDIN.2008.4618151.
- Ayyildiz M and Çetinkaya K. Comparison of four different heuristic optimization algorithms for the inverse kinematics solution of a real 4-DOF serial robot manipulator. *Neural Comput Appl* 2016; 27(4): 825–836.
- Mao B, Xie Z, Wang Y, et al. A hybrid differential evolution and particle swarm optimization algorithm for numerical kinematics solution of remote maintenance manipulators. *Fusion Eng Des* 2017; 124: 587–590.
- Mao B, Xie Z, Wang Y, et al. A hybrid strategy of differential evolution and modified particle swarm optimization for numerical solution of a parallel manipulator. *Math Probl Eng* 2018; 2018: 1–9.
- Ren ZW, Wang ZH, and Sun LN. A hybrid biogeography-based optimization method for the inverse kinematics problem of an 8-DOF redundant humanoid manipulator. *Front Inform Tech El* 2015; 16(7): 607–616.
- Bayati M. Using cuckoo optimization algorithm and imperialist competitive algorithm to solve inverse kinematics problem for numerical control of robotic manipulators. *Proc Institut Mech Eng I J Syst Control Eng* 2015; 229(5): 375–387.
- Gao X, Mu Y, and Gao Y. Optimal trajectory planning for robotic manipulators using improved teaching-learning-based optimization algorithm. *Ind Robot Int J* 2016; 43(3): 308–316.

20. Simon D. *Evolutionary optimization algorithms*. Hoboken: John Wiley & Sons, 2013.
21. Storn R and Price K. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *J Global Optim* 1997; 11(4): 341–359.
22. Atashpaz-Gargari E and Lucas C. Imperialist competitive algorithm: an algorithm for optimization inspired by imperialistic competition. In: *2007 IEEE congress on evolutionary computation*, Singapore, Singapore, 25-28 September 2007, pp. 4661–4667. IEEE. DOI: 10.1109/CEC.2007.4425083.
23. Santana-Castolo MH, Morales JA, Torres-Ramos S, et al. Study of particle swarm optimization algorithms using message passing interface and graphical processing units employing a high performance computing cluster. In: Gitler I and Klapp J (eds) *High Performance Computer Applications*. Cham: Springer International. 2016, pp. 116–131.
24. Meselhi MA, Elsayed SM, Essam DL, et al. Fast differential evolution for big optimization. In: *2017 11th international conference on software, knowledge, information management and applications (SKIMA)*, Malabe, Sri Lanka, 6–8 December 2017, pp. 1–6. IEEE. DOI: 10.1109/SKIMA. 2017. 8294137.
25. Derrac J, Garca S, Molina D, et al. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm Evol Comput* 2011; 1(1): 3–18.